University of Waterloo E-Thesis Template for LaTeX

by

Pat Neugraad

A thesis
presented to the University of Waterloo
in fulfillment of the
thesis requirement for the degree of
Doctor of Philosophy
in
Philosophy of Zoology

Waterloo, Ontario, Canada, 2023

© Pat Neugraad 2023

Examining Committee Membership

The following served on the Examining Committee for this thesis. The decision of the Examining Committee is by majority vote.

External Examiner:	Bruce Bruce Professor, Dept. of Philosophy of Zoology, University of Wallamaloo
Supervisor(s):	Ann Elk Professor, Dept. of Zoology, University of Waterloo Andrea Anaconda Professor Emeritus, Dept. of Zoology, University of Waterloo
Internal Member:	Pamela Python Professor, Dept. of Zoology, University of Waterloo
Internal-External Member:	Meta Meta Professor, Dept. of Philosophy, University of Waterloo
Other Member(s):	Leeping Fang Professor, Dept. of Fine Art, University of Waterloo

Author's Declaration

I hereby declare that I am the sole author of this thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I understand that my thesis may be made electronically available to the public.

Abstract

This is the abstract.

Acknowledgements

I would like to thank all the little people who made this thesis possible.

Dedication

This is dedicated to the one I love.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 An overview of TLS 1.3

Transport Layer Security (TLS) is a communication protocol with which two parties can exchange data in the presence of passive or active adversaries while preserving **confidentiality, integrity, and authenticity**. It was first developed at Netscape in 1994 and went through many iterations. The latest revision is TLS 1.3, formally standardized by IETF in RFC 8446 [Res18] in 2018. TLS 1.3 made significant improvements over TLS 1.2 [DR08] and prior versions, including deprecating weak symmetric cipher suites, requiring forward secrecy at all times, and restructuring the handshake state machine in order to simplify the handshake workflow. According to Cloudflare [Wes24], TLS 1.3 reached more than 93% adoption on the Internet. Given TLS 1.3's superior design and nearly universal adoption over other versions of TLS, this work will focus on this version unless otherwise specified.

Figure 1.1 illustrates a simple TLS 1.3 handshake over TCP. A new TLS 1.3 connection begins with a handshake in which the two parties exchange a specific sequence of messages to negotiate cryptographic parameters, establish shared secrets, and authenticate each other. If the handshake is successful, then the connection transitions to exchanging application data using the encryption scheme and secret keys obtained from the handshake phase. In TLS 1.3, the handshake can be further divided into two phases:

1. **Key exchange**

   (a) `ClientHello`: client publishes its supported symmetric cipher suites (AEAD,

hash functions), signature algorithms, a random nonce, and key exchange public keys.

(b) `ServerHello`: server chooses its preferred symmetric cipher suite, as well as publishes its own random nonce and key exchange public keys.

2. **Authentication**

(a) `Certificates`: server sends a chain of X.509 certificates that bind its identity to a digital signature public key; optionally, server can request client authentication (`CertificateRequest`), in which case the client must also send a chain of certificates.

(b) `CertificatesVerify`: server proves ownership fo the corresponding digital signature secret key by producing a signature over the handshake transcript; client will do the same upon server's request

(c) `Finished`: both parties prove to each other the integrity of the handshake by producing an HMAC over the handshake transcript

1.1.1 Certificate-based authentication

Authentication in TLS 1.3 relies on digital signatures and X.509 certificates. Figure 1.2 illustrates a certificate chain.

Each X.509 certificate encodes three pieces of information: identities of the issuer and the subject, a cryptographic public key, and a cryptographic signature over the identity and the public key. Through this signature, the issuer testifies the subject's ownership of the public key and binds the public key to the identity. The signature of the issuer is verified using the issuer's public key, which is usually presented via another certificate of identical format. This creates a chain of trust in which each certificate is verified by the public key in another certificate, up to the root of trust, which is usually a self-signed certificate whose signature is verified by the public key on the same certificate.

The *Public Key Infrastructure (PKI)* consists of entities that play different roles in this chain of trust. The server (and optionally client) is the leaf node of the chain, and the entities issuing certificates are called *Certificate Authorities (CA)*. The root of this chain is thus referred to as *root CA*, while other issuers between the leaf and the root are referred to as intermediate CAs. A selection of trusted root CA certificates are typically pre-installed in client devices by the vendors (e.g. Google, Mozilla, Apple, Microsoft, etc.).

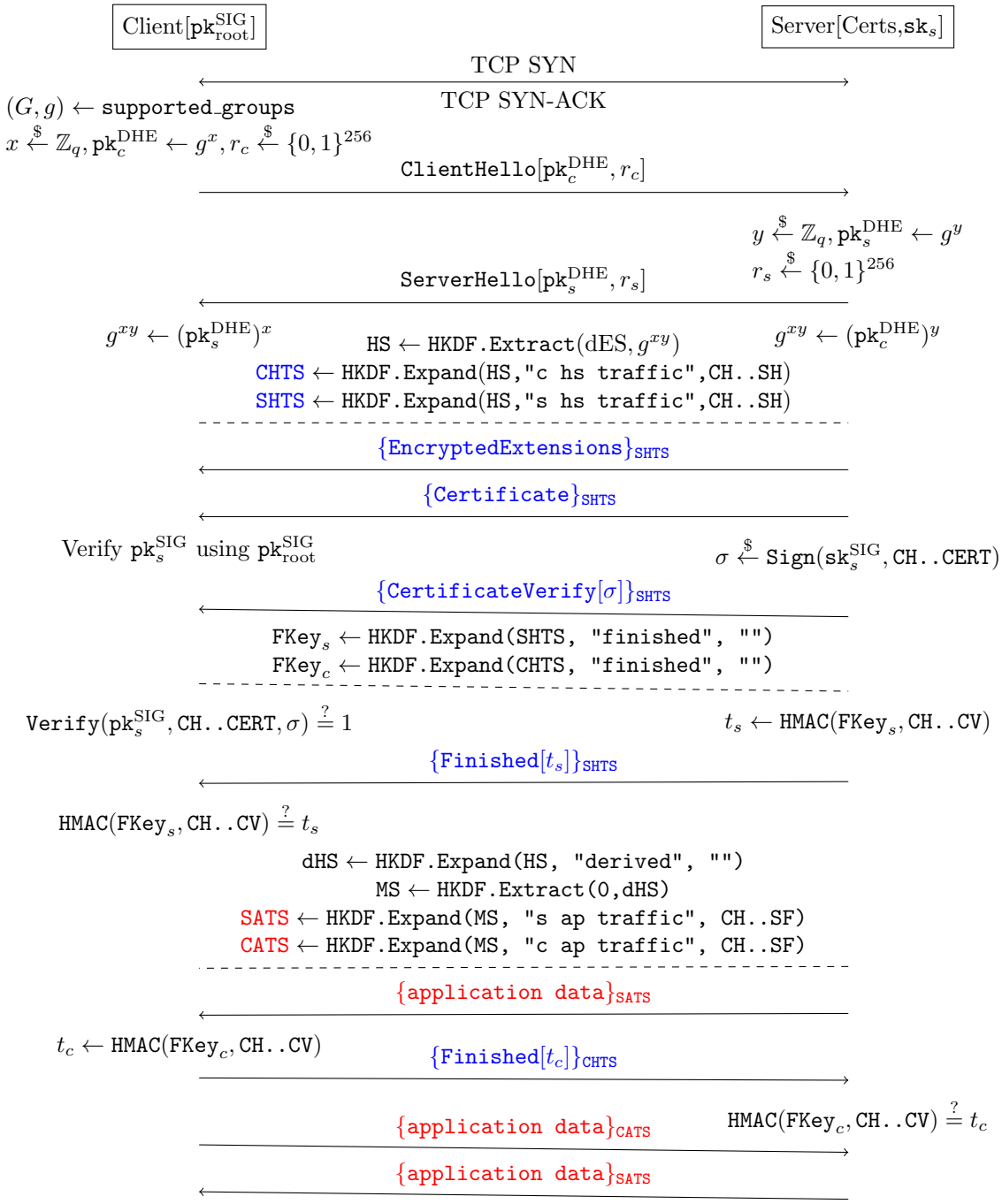

Figure 1.1: TLS 1.3 handshake with unilateral authentication

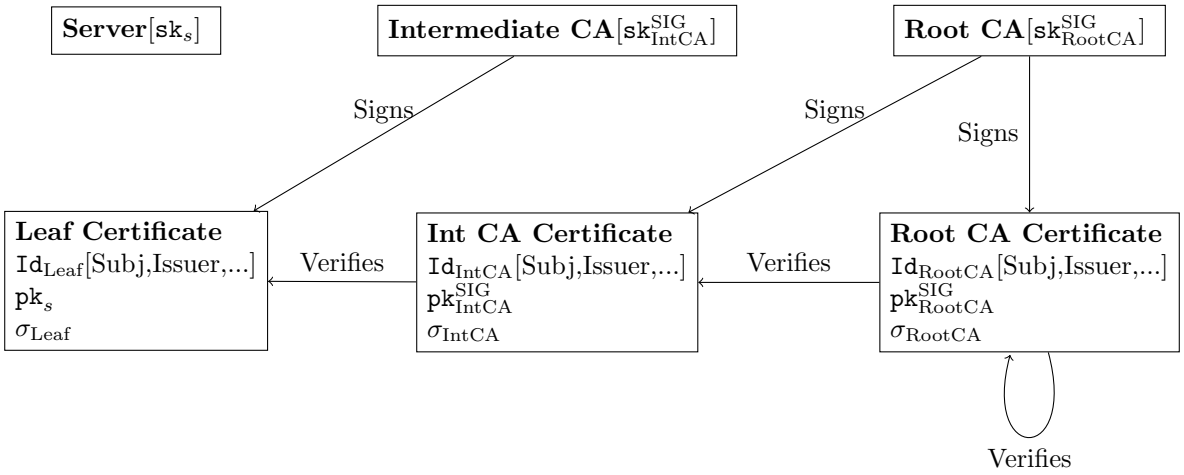

Figure 1.2: Certificate chain and PKI

1.1.2 Key schedule and HKDF

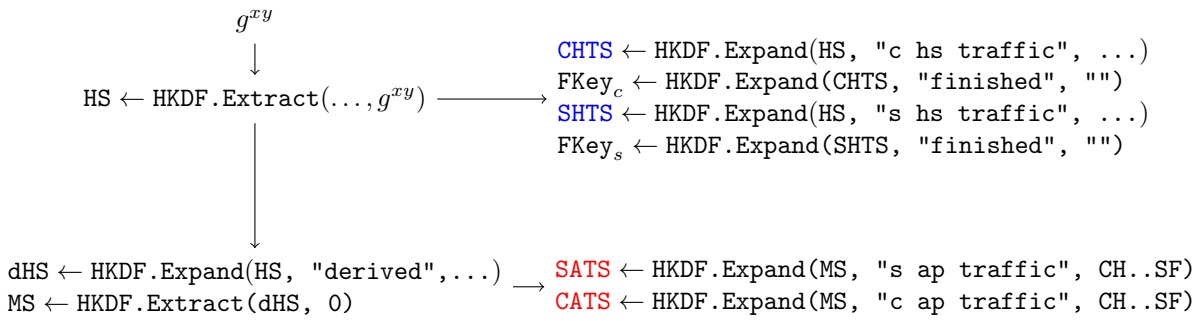

Figure 1.3: HKDF in TLS 1.3 key schedule

TLS 1.3 uses the HMAC-based Key Derivation Function (HKDF) [KE10] to derive cryptographic keys from input keying material (Figure 1.3). Using an extract-and-expand KDF is necessary because the n-bit shared secret obtained from public-key cryptography (e.g. g^{xy} in a Diffie-Hellman key exchange) is not a uniformly random n-bit string [Kra10]. Instead, it typically contains fewer bits of computational entropy. [1]

In TLS 1.3, the function `HKDF.Extract` serves as an entropy extractor that concentrates the available randomness in the input keying material. Another critical function of HKDF

[1] As stated in [Kra10], an adversary with knowledge of g^x and g^y can deterministically compute g^{xy}, implying that the statistical entropy of g^{xy} given g^x and g^y is zero.

in TLS 1.3 is to derive computationally independent keys via `HKDF.Expand`. As illustrated in Figure 1.1, all handshake messages following `ServerHello`, as well as all application data, are encrypted using different symmetric keys. Specifically, handshake messages are protected using a pair of handshake traffic keys—client handshake traffic secret (CHTS) and server handshake traffic secret (SHTS)—while application data is encrypted under a separate pair of application traffic keys—client application traffic secret (CATS) and server application traffic secret (SATS). Additionally, there is a unique symmetric key for each party to produce the HMAC tag used in their respective `Finished` messages. According to the HKDF model described in [KW16, Kra10], each of these keys—CATS, SATS, CHTS, SHTS, the client finished key, and the server finished key—is computationally independent from all others.

1.1.3 Security analysis

What are the security goals of TLS 1.3? How does TLS achieve them? Read and understand the formal analysis, summarize them here.

1.1.4 Transition to post-quantum

The security of TLS 1.3 was thoroughly analyzed from a theoretical perspect through a long line of work [DFGS15, DFGS16, BR93, FG14, DFGS21]. From a high level, TLS 1.3 aims to achieve confidentiality, integrity, authenticity, forward secrecy, and resistance to replay attacks in the presence of an adversary who controls the network, can passively eavesdrop and/or actively alter the communication, can compromise long-term asymmetric private keys, and can compromise some derived session keys[2], all across multiple sessions. The protocol's ability to achieve this goal depends on the strength of the various cryptographic primitives: the entropy of the nonces (client and server's `random` in the `Hello` messages), the collision resistance of the hash function (used for maintaining handshake transcript), the order of the ephemeral Diffie-Hellman group, the EUF-CMA security of the signature, the PRF security of the HKDF, the IND-CCA security of the AEAD, etc.

The advent of large-scale quantum computers affects all cryptographic aspects of the TLS 1.3 protocol. For example, Grover's quantum search algorithm [Gro96] would half the number of steps required to search through a key space, thereby halving the security

[2]The security model does not permit the adversary to directly compromise certain critical secrets and randomness, such as the Diffie-Hellman shared secret, since TLS 1.3 is not designed to be secure if these secrets are compromised

of symmetric ciphers. Brassard et al. [BHT97] presented a quantum algorithm that finds hash function collisions in $O(2^{\frac{n}{3}})$ hash queries instead of $O(2^{\frac{n}{2}})$ queries in the classical setting. Fortunately, the asymptotic security of symmetric primitives remains unchanged to this day, so the quantum threat can be straightforwardly mitigated by adopting "bigger" parameters, such as upgrading AES-128 to AES-256, and upgrading SHA256 to SHA512. On the other hand, all asymmetric primitives used in TLS 1.3 (ECDHE, RSA, ECDSA, and EdDSA) are based on hard number-theoretic problems, which Peter Shor's quantum algorithm [Sho94, Sho97] can solve with polynomial complexity. Therefore, using bigger parameters is not sufficient for making the public-key primitives of TLS 1.3 quantum resistant; instead, entirely new classes of cryptographic schemes for which no efficient quantum algorithm is known are required. Due to the "harvest-now-decrypt-later" attacks, adopting quantum-resistant key exchange algorithms takes the highest urgency, although due to the need for proven security in real-world usage, the transition to entirely post-quantum cryptography must start as soon as possible.

At the time of writing this thesis, the NIST post-quantum cryptography standardization project has standardized one key encapsulation mechanism based on hard lattice problems (ML-KEM) and two digital signature algorithms, one based on lattice problems (ML-DSA) and one based on hash functions (SLH-DSA). There are two more algorithms (Falcon and HQC) that are currently going through the standardization process.

1.2 Post-quantum IoT security

1.2.1 Software optimization

pqm4 [KRSS19].

1.2.2 Hardware accelerations

Because embedded devices have limited computing capacity compared to desktop processors, hardware acceleration is a popular strategy for implementing cryptographic protocols on embedded devices. In this section we briefly describe some popular strategies for accelerating cryptographic operations in constrained environments.

Accelerating symmetric operations. Hardware acceleration for symmetric primitives has been widely available to modern processors. Dedicated AES instructions were first

brought to Intel processors with the Westmere microarchitecture in 2010 [3]. On embedded devices, ARMv8's Cryptography Extensions included hardware-accelerated AES/SHA256 instructions in 2011 [4]. The widespread availability of hardware-accelerated symmetric primitives motivated many post-quantum cryptographic schemes to include variants that use AES/SHA256 as alternative to Keccak for implementing pseudorandom functions (PRF) and pseudorandom generators (PRG). For example, Kyber's round 2 submission included a "90s-variant" that used AES in counter mode and SHA2 instead of Keccak, and optimized implementation of the 90s-variant offered significant performance improvements over the optimized implementation of "standard" variant using Keccak [**?**]. Similarly, hash-based signature SPHINCS+ also offered variants that used SHA2 instead of Keccak [**?**].

Accelerating modular arithmetics. A second strategy for accelerating post-quantum cryptography is to target the modular arithmetics. Besides hardware accelerations dedicated to specific PQC scheme [HOKG18, KAK18, ACZ18], there are also active efforts at re-purposing RSA/ECC hardware accelerators [AHH+19]. Banerjee et al. [BDC20] combined hardware acceleration for both symmetric primitives and modular arithmetics and achieved order-of-magnitude speed and energy efficiency improvements over software-only implementations.

1.2.3 Authenticating the embedded device

Prior works [TLF+21b, BKNS20b, GW22] evaluating post-quantum TLS on embedded systems primarily focused on configurations in which the constrained device is an anonymous TLS client who does not need to authenticate itself to the server. While the TLS 1.3 (and KEMTLS) supports mutual authentication, implementing it on an embedded device posts unique challenges. The same set of challenges also apply to using embedded device as TLS server.

The primary obstacle to implementing authentication on embedded device is the threat of physical intrusion. For example, servers in a data center are physically secured within a building guarded by human with guns, so they often do not require extra hardware security measures. In some critical use cases where extra security is required (e.g. banking), a hardware security module (HSM) can be deployed on premise. By comparison, em-

[3] Citation needed

[4] http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.ddi0500e/CJHDEBAF.html

bedded devices are commonly deployed in unprotected if not hostile environments, where adversaries can easily capture the device and execute sophisticated physical attacks.

There are numerous hardware and software measures that can be deployed to protect secrets on an embedded devices, including but not limited to physical unclonable functions (PUF), trusted platform modules (TPM), cryptocoprocessors, and secure boot. In addition to cost and power consumption concerns, however, the existence of a "secure storage" means one can simply deploy a 256-bit symmetric key and use symmetric primitives (i.e. HMAC) to authenticate the device. In fact, TLS 1.3 supports handshake using pre-shared key (PSK)[Res18] in environments with no public key infrastructure (PKI). PSK automatically provides mutual authentication and also offers vastly superior performance by eliminating all asymmetric cryptographic operations. In other words, regardless of whether private keys can be securely stored or not, there is no scenario in which authenticating an embedded device using public-key cryptography makes sense.

Chapter 2

Optimizing post-quantum TLS

This section, we introduce two optimization on post-quantum TLS: Section 2.1 introduces KEMTLS, and Section 2.2 introduces IND-1CCA KEM for ephemeral key exchange.

2.1 KEMTLS

KEMTLS [SSW20], first proposed by Schwabe, Stebila, and Wiggers in 2020, is a variation on TLS 1.3 that uses key encapsulation mechanism (KEM) instead of digital signature for peer authentication. KEMTLS is motivatd by the observation that post-quantum signature candidates generally have larger public key and signature than post-quantum KEM candidates' public key and ciphertext, so by using KEMs instead of signatures, one can obtain significant savings in communication sizes.

Section 2.1.1 will introduce the KEMTLS handshake workflow. Section 2.1.2 will review the security properties of KEMTLS and compare them with TLS 1.3.

2.1.1 KEMTLS Handshake

Figure 2.1 illustrates a KEMTLS handshake with unilateral authentication. Like TLS 1.3's handshake (Figure 1.1), KEMTLS handshake can be cleanly divided into a key exchange phases where the two parties establish an ephemeral shared secret, and an authentication phase where the server proves its identity to the client. However, KEMTLS' authentication flow diverges significantly from TLS 1.3's authentication flow, and there are subtle downstream differences. We will discuss them in detail in this section.

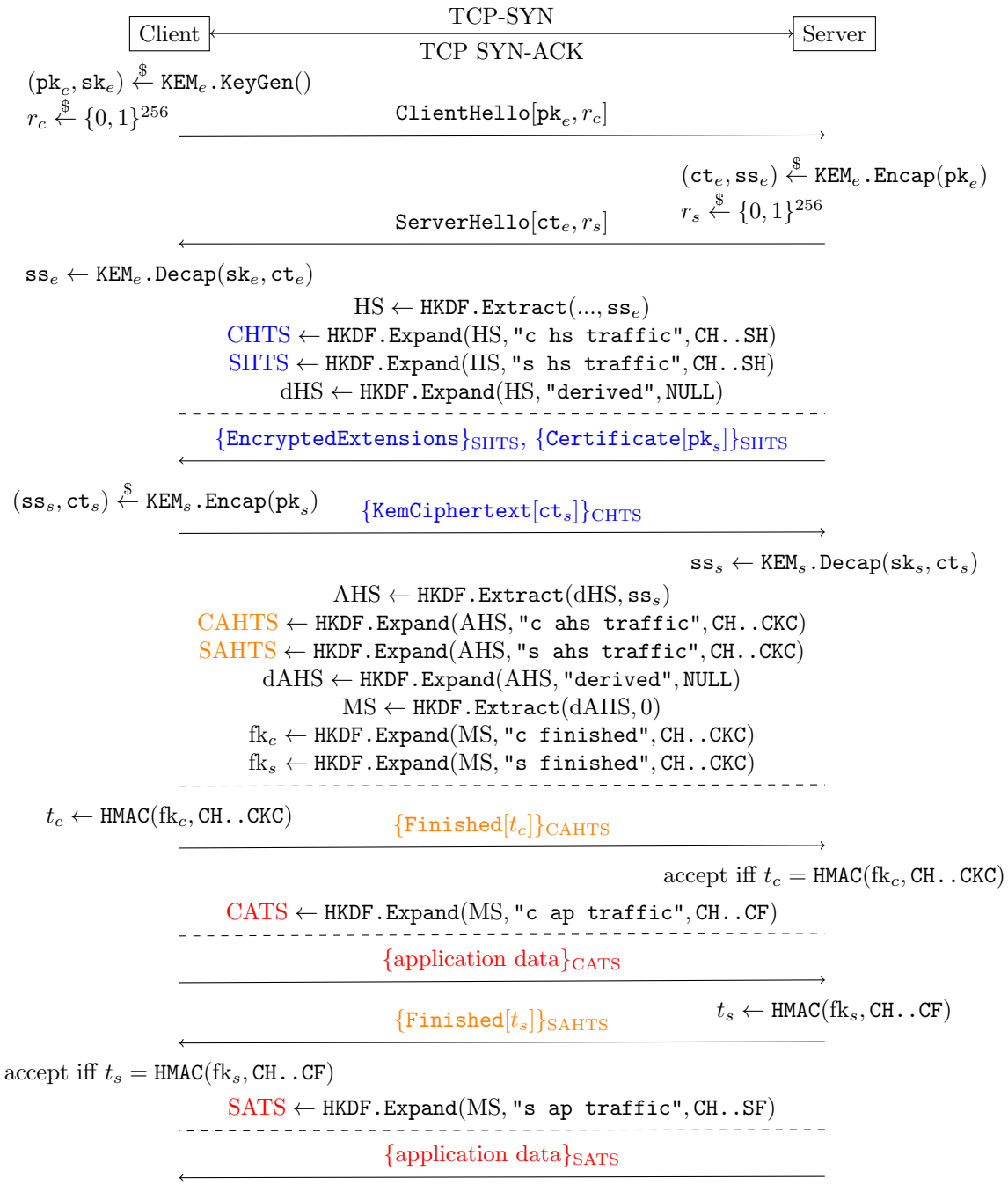

Figure 2.1: KEMTLS handshake with unilateral authentication

Ephemeral key exchange. Like TLS 1.3, a KEMTLS handshake begins with establishing a TCP stream and client/server exchanging `ClientHello` and `ServerHello` in this order. Client first picks a KEM scheme $\texttt{KEM}_e$ for ephemeral key exchange, then generates the ephemeral keypair $(\texttt{pk}_e, \texttt{sk}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.KeyGen}()$. The ephemeral public key $\texttt{pk}_e$ is sent to the server in `ClientHello`, which also includes a 32-byte client nonce $r_c \in \{0,1\}^{256}$, client's supported cipher suites, and other parameters not crucial to the workflow of KEMTLS. If server accepts the parameters in `ClientHello`, then it encapsulates a random secret $(\texttt{ct}_e, \texttt{ss}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.Encap}(\texttt{pk}_e)$. The ciphertext $\texttt{ct}_e$ is included in `ServerHello` sent to the client, alongside server's random nonce $r_s \in \{0,1\}^{256}$ and other parameters. Finally, after receiving `ServerHello`, client recovers the ephemeral shared secret by decapsulating the ciphertext $\texttt{ss}_e \leftarrow \texttt{KEM}_e\texttt{.Decap}(\texttt{sk}_e, \texttt{ct}_e)$.

At this stage, client and server have established an ephemeral shared secret $\texttt{ss}_e$ and will update their key schedule synchronously. Specifically, each extracts a handshake secret (HS) from $\texttt{ss}_e$ using `HKDF.Extract`, then derive client handshake traffic secret (CHTS), server handshake traffic secret (SHTS), and derived handshake secret (dHS) using `HKDF.Expand` under different labels. CHTS and SHTS will be used to encrypt subsequent handshake messages from the client and the server respectively. dHS on the other hand serves as the secret state of the key schedule and will participate in subsequent key schedule update. Figure 2.1 used color-coding and subscripts to clarify which handshake message is encrypted and under which secret.

(Implicit) Server authentication. Like TLS 1.3, the authentication phase begins with server sending `Certificate`, but the workflow diverges afterwards. With KEMTLS, server's long-term public key $\texttt{pk}_s$ is a KEM encapsulation key. To prove server's possession of the corresponding decapsulation key, client uses server's long-term public key to encapsulate a secret $(\texttt{ct}_s, \texttt{ss}_s) \xleftarrow{\$} \texttt{KEM}_s\texttt{.Encap}(\texttt{pk}_s)$, then sends the ciphertext in a handshake message $\texttt{KemCiphertext}[\texttt{ct}_s]$. After receiving $\texttt{KemCiphertext}[\texttt{ct}_s]$, server recovers the authentication secret using its long-term secret key $\texttt{ss}_s \leftarrow \texttt{KEM}_s\texttt{.Decap}(\texttt{sk}_s, \texttt{ct}_s)$.

After this round of messages, the two parties have obtained another shared secret $\texttt{ss}_s$ and will perform a second update to the key schedule. First, the (unauthenticated) handshake secret (HS) will be upgraded to an authenticated handshake secret (AHS) by extracting randomness from dHS and $\texttt{ss}_s$. From AHS, the handshake traffic secrets will be upgraded to authenticated handshake traffic secrets (CAHTS, SAHTS). The key schedule's secret state is updated from AHS, then used to produce the Master Secret (MS). Last but not least, the `Finished` keys fk_c, fk_s are derived from MS.

Key confirmation/explicit authentication. The final round of handshake message includes client and server exchanging `Finished`. Like TLS 1.3, the `Finished` message contains an HMAC tag over the handshake transcript up to the point when the message is sent. Also like TLS 1.3, each party can derive the appropriate application traffic keys (CATS, SATS) and start sending application data right after sending its own `Finished`. On the other hand, unlike TLS 1.3, the order of "who first sends `Finished` and application" is flipped: where in TLS 1.3, server sends `Finished` before client, in KEMTLS, client sends `Finished` before server. This means that from client's perspective, TLS 1.3 and KEMTLS require the same one round trip (1-RTT) of handshake messages before client can start sending application data. From server's perspective, the number of round trips required before sending application data increased, but as [SSW20] pointed out, in most TLS applications, client sends the first round of application data, so the increase of round trips for server is likely irrelevant to the latency of the application. The flipped order of `Finished` also has security implications, since client will be sending its first round of application data before having received server's `Finished`. We will discuss them in details in Section 2.1.2.

2.1.2 KEMTLS security

KEMTLS has identical security goals to TLS 1.3. Analysis of KEMTLS' security thus follows the same framework as the formal analysis of TLS 1.3's security described in Section 1.1.3. A "pen-and-paper" proof of KEMTLS security properties is first presented in [SSW20]. Later, a computer-verified symbolic analysis of KEMTLS was conducted using Tamarin [CHSW22, MSCB13]. At a high level, KEMTLS achieves confidentiality, integrity, authenticity, forward secrecy, and resistance to replays if the client/server nonces are sufficiently large, the ephemeral key exchange KEM is IND-1CCA secure, the authentication KEM is IND-CCA secure, the chosen hash function is collision resistant, the HKDF is a secure PRF, and the AEAD is IND-CCA secure. These properties are formally stated by Theorem 2.1.1:

Theorem 2.1.1 (Multi-stage security of KEMTLS [SSW20])**.** *Let n be the number of sessions and U be the number of parties. The advantage of any probabilistic polynomial-time adversary $\mathcal{A}$ in breaking the multi-stage security of KEMTLS is bounded by:*

$$\mathit{Adv}(\mathcal{A}) \leq \frac{n^2}{2^{|r_c|}} + \epsilon_H^{COL} + 6n\Bigg(n(\epsilon_{KEM_e}^{IND\text{-}1CCA} + \epsilon_{HKDF.Ext}^{PRF} + 2\epsilon_{HKDF.Ext}^{dual\text{-}PRF} + 3\epsilon_{HKDF.Exp}^{PRF} + \epsilon_{HMAC}^{EUF\text{-}CMA}) + U(\epsilon_{KEM_s}^{IND\text{-}CCA} + 2\epsilon_{HKDF.Ext}^{dual\text{-}PRF} + 2\epsilon_{HKDF.Exp}^{PRF} + \epsilon_{HMAC}^{EUF\text{-}CMA})\Bigg)$$

Implicit authentication. In TLS 1.3, client will not receive server's application data nor send its own application data until after it has received and validated server's `Finished`, which means that all application data exchanged are explicitly authenticated. In KEMTLS, however, client can start sending application data after sending its `Finished` but before receiving server's `Finished`. The application data sent before receiving server's `Finished` are encrypted under client's application traffic key, which contains entropy from the authentication secret $\texttt{ss}_s$. This means that, if $\texttt{KEM}_s$ is IND-CCA secure, then no party without the long-term seret key $\texttt{sk}_s$ can correctly obtain the AEAD secret key. In other words, the early application data is **implicitly authenticated**. While this provides less security than explicitly authenticated application data, to this day there is no known attack exploiting the implicit authentication. In addition, explicit authentication can be achieved at one more round trip, at which time KEMTLS becomes as secure as TLS 1.3 using comparable cryptographic primitives.

2.2 IND-1CCA KEM

INDistinguishability under (adaptive) **C**hosen **C**iphertext **A**ttacks (IND-CCA) is the desired level of security for a key encapsulation mechanism. Intuitively, IND-CCA security requires that the shared secret from a random encapsulation cannot be distinguished from random bit string of equal length with non-negligible probability, even when the adversary has access to a decapsulation oracle. Theoretically speaking, the number of decapsulation queries is bounded by the requirement that the adversary is "efficient": that is, the adversary, classical or quantum, can only have polynomial time complexity. In practice, NIST's PQC standardization project evaluates a candidate KEM's CCA security under the upper bound of 2^{64} decapsulation queries[Nat17].

However, within the context of an TLS 1.3 (and KEMTLS) key exchange, IND-CCA security is overkill. This is because TLS 1.3 is designed with forward secrecy, which requires that the keypair $(\texttt{pk}_e, \texttt{sk}_e) \xleftarrow{\$} \texttt{KEM}_e\texttt{.KeyGen}()$ exchanged in `ClientHello` and `ServerHello` be freshly generated at every session. Consequently, in a correct TLS 1.3 implementation, each private key will only be used to decapsulate at most one ciphertext. This naturally defines an alternative security definition called IND-1CCA, which is identical to IND-CCA except that the decapsulation oracle will answer at most one query. A similar notion of "security under one query" for Diffie-Hellman-style key exchange protocol called `snPRF-ODH` is defined in [DFGS21] and used in the security reduction of TLS 1.3 handshake protocol.

On the other hand, using IND-1CCA security brings meaningful performance improvements. A popular strategy for building an IND-CCA secure KEM is to start with an

IND-CPA secure public key encryption scheme (PKE), then apply some variation of the Fujisaki-Okamoto transformation [FO99, FO13, HHK17]. The key techniques for achieving chosen-ciphertext security in are *de-randomization* and *re-encryption*. While there is some contention on whether *de-randomiation* hurts security [Ber21] (which indirectly hurts performance because the same security level may require larger set of parameters), there is no doubt that *re-encryption* hurts performance. The performance penalty of re-encryption is especially pronounced with many lattice-based cryptosystems whose CPA-secure encryption sub-routine is much slower than the CPA-secure decryption sub-routine. As we will show in the following section, IND-1CCA constructions can preserve randomization of the underlying PKE while not requiring re-encryption.

In Section 2.2.1 we will present an original IND-1CCA construction that transforms a CPA-secure PKE into an IND-1CCA secure KEM. Section 2.2.2 then surveys and discusses other constructions.

2.2.1 The encrypt-then-MAC transformation

Let $\mathcal{B}^*$ denote the set of finite bit strings. Let $\mathcal{K}_{\texttt{KEM}}$ denote the set of all possible shared secrets. Let $(\texttt{KeyGen}_{\texttt{PKE}}, \texttt{Enc}_{\texttt{PKE}}, \texttt{Dec}_{\texttt{PKE}})$ be a PKE defined over message space $\mathcal{M}_{\texttt{PKE}}$ and ciphertext space $\mathcal{C}_{\texttt{PKE}}$. Let $\texttt{MAC} : \mathcal{K}_{\texttt{MAC}} \times \mathcal{B}^* \to \mathcal{T}$ be a MAC over key space $\mathcal{K}_{\texttt{MAC}}$ and tag space $\mathcal{T}$. Let $G : \mathcal{B}^* \to \mathcal{K}_{\texttt{MAC}}, H : \mathcal{B}^* \to \mathcal{K}_{\texttt{KEM}}$ be hash functions. The encrypt-then-MAC transformation $\texttt{ETM}[\texttt{PKE}, \texttt{MAC}, G, H]$ constructs a KEM $(\texttt{KeyGen}_{\texttt{ETM}}, \texttt{Encap}_{\texttt{ETM}}, \texttt{Decap}_{\texttt{ETM}})$ (Figure 2.2).

We chose to construct $\texttt{KEM}_{\texttt{ETM}}$ using implicit rejection $K \leftarrow H(s, c)$: on invalid ciphertexts, the decapsulation routine returns a fake shared secret that depends on the ciphertext and some secret values, though choosing to use explicit rejection should not impact the security of the KEM. In addition, because the underlying PKE can be randomized, the shared secret $K \leftarrow H(m, c)$ must depend on both the plaintext and the ciphertext. According to [Den03, HHK17], if the input PKE is *rigid* (i.e. $m = \texttt{Dec}(\texttt{sk}, c)$ if and only if $c = \texttt{Enc}(\texttt{pk}, m)$, such as with RSA), then the shared secret may be derived from the plaintext alone $K \leftarrow H(m)$.

The CCA security of $\texttt{KEM}_{\texttt{ETM}}$ can be intuitively argued through an adversary's inability to learn additional information from the decapsulation oracle. For an adversary A to produce a valid tag for some unauthenticated ciphertext c', it must either know the correct symmetric key or produce a forgery. Under the Random Oracle Model (ROM), A cannot know the symmetric key without knowing its pre-image under the hash function G, so A must either produced c' honestly, or have broken the one-wayness of the underlying PKE.

$\mathtt{KeyGen}_{\mathtt{ETM}}()$

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}_{\mathtt{PKE}}()$
2: $s \xleftarrow{\$} \mathcal{M}_{\mathtt{PKE}}$
3: $\mathtt{sk} \leftarrow (\mathtt{sk}, s)$
4: **return** $(\mathtt{pk}, \mathtt{sk})$

$\mathtt{Encap}_{\mathtt{ETM}}(\mathtt{pk})$

1: $m \xleftarrow{\$} \mathcal{M}_{\mathtt{PKE}}$
2: $k \leftarrow G(m)$
3: $c' \xleftarrow{\$} \mathtt{Enc}_{\mathtt{PKE}}(\mathtt{pk}, m)$
4: $t \leftarrow \mathtt{MAC}(k, c')$
5: $c \leftarrow (c', t)$
6: $K \leftarrow H(m, c)$
7: **return** (c, K)

$\mathtt{Decap}_{\mathtt{ETM}}(\mathtt{sk}, c)$

Require: $c = (c', t)$
Require: $\mathtt{sk} = (\mathtt{sk}', s)$
1: $\hat{m} \leftarrow \mathtt{Dec}_{\mathtt{PKE}}(\mathtt{sk}', c')$
2: $\hat{k} \leftarrow G(\hat{m})$
3: **if** $\mathtt{MAC}(\hat{k}, c') = t$ **then**
4: $\quad K \leftarrow H(\hat{m}, c)$
5: **else**
6: $\quad K \leftarrow H(s, c)$
7: **end if**
8: **return** K

Figure 2.2: The encrypt-then-MAC KEM routines

This means that the decapsulation oracle will not leak information on decryption that the adversary does not already know. We formalize the security in Theorem 2.2.1

Theorem 2.2.1. *For every IND-qCCA adversary A against $\mathit{KEM}_{\mathit{ETM}}$ making q decapsulation queries, there exists a OW-PCA adversary B against the underlying PKE making at least q decapsulation queries, and an existential forgery adversary C against the underlying MAC such that:*

$$\mathit{Adv}^{\mathit{IND\text{-}CCA}}_{\mathit{KEM}_{\mathit{ETM}}}(A) \leq q \cdot \mathit{Adv}_{\mathit{MAC}}(C) + 2 \cdot \mathit{Adv}^{\mathit{OW\text{-}PCA}}_{\mathit{PKE}}(B)$$

From here, we can reduce OW-PCA security to OW-CPA security using a guessing argument: a CPA adversary can simulate a PCA oracle simply by randomly sampling from $\{0, 1\}$ as response to every plaintext-checking query. Thus, the probability that all guesses are correct and PCA adversary retains its advantage is at most 2^{-q}. In other words:

Lemma 2.2.2. *For every efficient q-query OW-PCA adversary A against some PKE, there exists some OW-CPA adversary B such that*

$$\mathit{Adv}^{\mathit{OW\text{-}PCA}}_{\mathit{PKE}}(A) \leq 2^q \cdot \mathit{Adv}^{\mathit{OW\text{-}CPA}}_{\mathit{PKE}}(B)$$

Proof of Theorem 2.2.1

We will prove Theorem 2.2.1 using a sequence of game. A summary of the the sequence of games can be found in Figure 2.3 and 2.4. From a high level we made three incremental modifications to the IND-CCA game for $\texttt{KEM}_{\texttt{ETM}}$:

1. Replace the true decapsulation oracle with a simulated decapsulation oracle. The simulated decapsulation oracle does not directly decrypt the queried ciphertext. Instead it searches through the hash oracle and looks for matching queries using the plaintext-checking oracle. The true decapsulation oracle and the simulated decapsulation oracle disagree if and only if the adversary queries with a ciphertext that contains a forged MAC tag, so the adversary cannot distinguish the two games more than it can perform existential forgery against the underlying MAC.

2. Replace the pseudorandom MAC key $k^* \leftarrow G(m^*)$ with a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$. Under ROM, the adversary cannot distinguish this game from the previous one unless it queries the hash oracle with m^*, in which case a second adversary with access to a plaintext-checking oracle can win the OW-PCA game against the underlying PKE.

3. Replace the pseudorandom shared secret $K_0 \leftarrow H(m^*, c)$ with a truly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$. Similar to the point above, the adversary cannot distinguish the change more than it can break the one-wayness of the underlying PKE. Furthermore, since both K_0 and K_1 are uniformly random, no adversary can have any advantage.

A OW-PCA adversary can then simulate the modified IND-CCA game for the KEM adversary, and the advantage of the OW-PCA adversary is associated with the probability of certain behaviors of the KEM adversary.

Proof. Game 0 is the standard KEM IND-CCA game. The decapsulation oracle $\mathcal{O}^{\texttt{Decap}}$ executes the decapsulation routine using the challenge keypair and return the results faithfully. The queries made to the hash oracles $\mathcal{O}^G, \mathcal{O}^H$ are recorded to their respective tapes $\mathcal{L}^G, \mathcal{L}^H$.

Game 1 is identical to game 0 except that the true decapsulation oracle $\mathcal{O}^{\texttt{Decap}}$ is replaced with a simulated oracle $\mathcal{O}_1^{\texttt{Decap}}$. Instead of directly decrypting c' as in the decapsulation routine, the simulated oracle searches through the tape $\mathcal{L}^G$ to find a matching query $(\tilde{m}, \tilde{k})$ such that $\tilde{m}$ is the decryption of c'. The simulated oracle then uses $\tilde{k}$ to validate the tag t against c'.

$\texttt{IND-CCA}$ game for $\texttt{KEM}_{\texttt{ETM}}$

1: $(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}_{\texttt{ETM}}()$
2: $m^* \xleftarrow{\$} \mathcal{M}$
3: $c' \xleftarrow{\$} \texttt{Enc}_{\texttt{PKE}}(\texttt{pk}, m^*)$
4: $k^* \leftarrow G(m^*)$ ▷ Game 0-1
5: $k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$ ▷ Game 2-3
6: $t \leftarrow \texttt{MAC}(k^*, c')$
7: $c^* \leftarrow (c', t)$
8: $K_0 \leftarrow H(m^*, c^*)$ ▷ Game 0-2
9: $K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$ ▷ Game 3
10: $K_1 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$
11: $b \xleftarrow{\$} \{0, 1\}$
12: $\hat{b} \leftarrow A^{\mathcal{O}^{\texttt{Decap}}}(\texttt{pk}, c^*, K_b)$ ▷ Game 0
13: $\hat{b} \leftarrow A^{\mathcal{O}_1^{\texttt{Decap}}}(\texttt{pk}, c^*, K_b)$ ▷ Game 1-3
14: **return** $[\![\hat{b} = b]\!]$

Hash oracle $\mathcal{O}^G(m)$

1: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ **then**
2: **return** $\tilde{k}$
3: **end if**
4: $k \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$
5: $\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$
6: **return** k

Decap oracle $\mathcal{O}^{\texttt{Decap}}(c)$

1: $(c', t) \leftarrow c$
2: $\hat{m} = \texttt{Dec}_{\texttt{PKE}}(\texttt{sk}', c')$
3: $\hat{k} \leftarrow G(\hat{m})$
4: **if** $\texttt{MAC}(\hat{k}, c') = t$ **then**
5: $K \leftarrow H(\hat{m}, c)$
6: **else**
7: $K \leftarrow H(z, c)$
8: **end if**
9: **return** K

$\mathcal{O}_1^{\texttt{Decap}}(c)$

1: $(c', t) \leftarrow c$
2: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = \texttt{Dec}_{\texttt{PKE}}(\texttt{sk}', c') \wedge \texttt{MAC}(\tilde{k}, c') = t$ **then**
3: $K \leftarrow H(\tilde{m}, c)$
4: **else**
5: $K \leftarrow H(z, c)$
6: **end if**
7: **return** K

$\mathcal{O}^H(m, c)$

1: **if** $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ **then**
2: **return** $\tilde{K}$
3: **end if**
4: $K \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$
5: $\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$
6: **return** K

Figure 2.3: Sequence of games in the proof of Theorem 2.2.1

$B(\mathtt{pk}, c'^*)$

1: $z \xleftarrow{\$} \mathcal{M}$
2: $k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
3: $t \leftarrow \mathtt{MAC}(k, c'^*)$
4: $c^* \leftarrow (c'^*, t)$
5: $K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
6: $\hat{b} \leftarrow A^{\mathcal{O}_B^{\mathtt{Decap}}, \mathcal{O}_B^G, \mathcal{O}_B^H}(\mathtt{pk}, c^*, K)$
7: **if** $\mathtt{ABORT}(m)$ **then**
8: **return** m
9: **end if**

$\mathcal{O}_B^H(m, c)$

if $\mathcal{O}^{\mathtt{PCO}}(m, c'^*) = 1$ **then**
$\mathtt{ABORT}(m)$
end if
if $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ **then**
return $\tilde{K}$
end if
$K \xleftarrow{\$} \mathcal{K}_{\mathtt{KEM}}$
$\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$
return K

$\mathcal{O}_B^{\mathtt{Decap}}(c)$

1: $(c', t) \leftarrow c$
2: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \mathcal{O}^{\mathtt{PCO}}(\tilde{m}, c') = 1 \wedge \mathtt{MAC}(\tilde{k}, c') = t$ **then**
3: $K \leftarrow H(\tilde{m}, c)$
4: **else**
5: $K \leftarrow H(z, c)$
6: **end if**
7: **return** K

$\mathcal{O}_B^G(m)$

1: **if** $\mathcal{O}^{\mathtt{PCO}}(m, c'^*) = 1$ **then**
2: $\mathtt{ABORT}(m)$
3: **end if**
4: **if** $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ **then**
5: **return** $\tilde{k}$
6: **end if**
7: $k \xleftarrow{\$} \mathcal{K}_{\mathtt{MAC}}$
8: $\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$
9: **return** k

Figure 2.4: OW-PCA adversary B simulates game 3 for IND-CCA adversary A in the proof for Theorem 2.2.1

If the simulated oracle accepts the queried ciphertext as valid, then there is a matching query that also validates the tag, which means that the queried ciphertext is honestly generated. Therefore, the true oracle must also accept the queried ciphertext. On the other hand, if the true oracle rejects the queried ciphertext, then the tag is simply invalid under the MAC key $k = G(\texttt{Dec}(\texttt{sk}', c'))$. Therefore, there could not have been a matching query that also validates the tag, and the simulated oracle must also rejects the queried ciphertext.

This means that from the adversary A's perspective, game 1 and game 0 differ only when the true oracle accepts while the simulated oracle rejects, which means that t is a valid tag for c' under $k = G(\texttt{Dec}(\texttt{sk}', c'))$, but k has never been queried. Under the random oracle model, such k is a uniformly random sample of $\mathcal{K}_{\texttt{MAC}}$ that the adversary does not know, so for A to produce a valid tag is to produce a forgery against the MAC under an unknown and uniformly random key. Therefore, we can bound the probability that the true decapsulation oracle disagrees with the simulated oracle by the probability that some MAC adversary produces a forgery:

$$P\left[\mathcal{O}^{\texttt{Decap}}(c) \neq \mathcal{O}_1^{\texttt{Decap}}(c)\right] \leq \texttt{Adv}_{\texttt{MAC}}(C).$$

Across all q decapsulation queries, the probability that at least one query is a forgery is thus at most $q \cdot P\left[\mathcal{O}^{\texttt{Decap}}(c) \neq \mathcal{O}_1^{\texttt{Decap}}(c)\right]$. By the difference lemma:

$$|\texttt{Adv}_{G_0}(A) - \texttt{Adv}_{G_1}(A)| \leq q \cdot \texttt{Adv}_{\texttt{MAC}}(C).$$

Game 2 is identical to game 1, except that the challenger samples a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\texttt{MAC}}$ instead of deriving it from m^*. From A's perspective the two games are indistinguishable, unless A queries G with the value of m^*. Denote the probability that A queries G with m^* by $P[\texttt{QUERY G}]$, then:

$$|\texttt{Adv}_{G_1}(A) - \texttt{Adv}_{G_2}(A)| \leq P\left[\texttt{QUERY G}\right].$$

Game 3 is identical to game 2, except that the challenger samples a uniformly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\texttt{KEM}}$ instead of deriving it from m^* and t. From A's perspective the two games are indistinguishable, unless A queries H with $(m^*, \cdot)$. Denote the probability that A queries H with $(m^*, \cdot)$ by $P[\texttt{QUERY H}]$, then:

$$|\texttt{Adv}_{G_2}(A) - \texttt{Adv}_{G_3}(A)| \leq P\left[\texttt{QUERY H}\right].$$

Since in game 3, both K_0 and K_1 are uniformly random and independent of all other variables, no adversary can have any advantage: $\mathtt{Adv}_{G_3}(A) = 0$.

We will bound $P[\mathtt{QUERY\ G}]$ and $P[\mathtt{QUERY\ H}]$ by constructing a OW-PCA adversary B against the underlying PKE that uses A as a sub-routine. B's behaviors are summarized in Figure 2.4.

B simulates game 3 for A: upon receiving the public key $\mathtt{pk}$ and challenge encryption c'^*, B samples random MAC key and session key to produce the challenge encapsulation, then feeds it to A. When simulating the decapsulation oracle, B uses the plaintext-checking oracle to look for matching queries in $\mathcal{L}^G$. When simulating the hash oracles, B uses the plaintext-checking oracle to detect when $m^* = \mathtt{Dec}(\mathtt{sk}', c'^\star)$ has been queried. When m^* is queried, B terminates A and returns m^* to win the OW-PCA game. In other words:

$$P\left[\mathtt{QUERY\ G}\right] \leq \mathtt{Adv}^{\mathtt{OW\text{-}PCA}}_{\mathtt{PKE}}(B),$$
$$P\left[\mathtt{QUERY\ H}\right] \leq \mathtt{Adv}^{\mathtt{OW\text{-}PCA}}_{\mathtt{PKE}}(B).$$

Combining all equations above produce the desired security bound. □

2.2.2 Related works

To our knowledge, the concept of IND-qCCA security is first published by Cramer et al.[CHH+07], whose proposed a generic black-box IND-qCCA PKE construction whose security reduces to the IND-CPA security of the underlying PKE and the existential unforgeability of the underlying one-time signature [LAM79]. It is particularly remarkable how this reduction is proved in the standard model instead of the random oracle model, although this came at the cost of computational efficiency.

KeyGen

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}^{\mathtt{PKE}}()$

Encap(pk)

1: $m \xleftarrow{\$} \mathcal{M}^{\mathtt{PKE}}$
2: $c \xleftarrow{\$} \mathtt{Enc}^{\mathtt{PKE}}(\mathtt{pk}, m)$
3: $K \leftarrow H(m, c)$
4: **return** (c, K)

Decap(sk, c)

1: $\hat{m} \leftarrow \mathtt{Dec}^{\mathtt{PKE}}(\mathtt{sk}, c)$
2: **if** $\hat{m} = \perp$ **then**
3: **return** $\perp$
4: **end if**
5: **return** $H(\hat{m}, c)$

Figure 2.5: T_H transformation

Huguenin-Dumittan and Vaudenay [HV22] extended this line of work to the post-quantum setting by proposing two IND-qCCA KEM constructions (respectively denoted by T_{CH} and T_H) from CPA-secure PKEs. T_{CH} achieves qCCA security using a "confirmation hash": the KEM ciphertext includes the PKE ciphertext and a hash of the PKE plaintext, and the KEM shared secret is derived from hashing the PKE plaintext (the encrypt-then-MAC transformatino can be seen as a special case of the T_{CH} transformation **not sure if we should mention this similarity**). The security reduction is loose by a factor of 2^q where q is the number of decapsulation queries. T_H achieves qCCA security by deriving the shared secret from hashing both the PKE plaintext and ciphertext (Figure 2.5), so decapsulation queries with modified ciphertext will be answered with independent random values under the rancom oracle model. T_H's construction is simpler than T_{CH} and retains the original ciphertext length, but on the other hand, the security reduction is looser, with a factor of q_H^{2q}, where q_H is the number of hash oracle queries, and q is the number of decapsulation queries. The authors also proved qCCA security of both transformations under quantum random oracle model (QROM).

In the same paper, the two authors also observed that in TLS 1.3, the handshake secret (and thus all subsequent secrets and keys) is derived with input from the handshake transcript, which means that the KEM public key and ciphertext, embedded in `ClientHello` and `ServerHello` respectively, are already hashed into the handshake secret. This is similar to the T_H construction where the shared secret is hashed from PKE plaintext and ciphertext, and indeed the authors proved that TLS 1.3 remains multi-stage secure even if the KEM is only CPA secure. The initial proof presented by the authors had a looseness factor of $O(q^6)$, which was later improved by Zhou et al.[ZJZ24] to $O(q)$ for randomized PKEs, and $O(1)$ for rigid PKEs.

Chapter 3

Implementing PQ-TLS and KEMTLS on embedded client

To evaluate post-quantum TLS 1.3 and KEMTLS on embedded clients, we implemented post-quantum TLS 1.3 and KEMTLS on top of WolfSSL[1] and using cryptographic primitives from PQClean[2][KSSW22]. Source code for this project is available on GithHub[3]. This chapter describes how we implemented post-quantum TLS 1.3 and KEMTLS.

3.1 WolfSSL

WolfSSL is a modern open-source TLS library written in the C programming language. In addition to a complete TLS stack, WolfSSL also includes a robust cryptogrpahy library called WolfCrypt. WolfSSL is optimized for code size, memory footprint, and portability, and WolfCrypt received many industry/government certification including FIPS140-3 from the US federal government and DO-178C DAL-A for avionics. There are other TLS/SSL libraries with support for embedded devices. For example, [BKNS20b] evaluated post-quantum TLS 1.2 using the open-source mbedTLS library [4]. Unfortunately mbedTLS did not support TLS 1.3 until December 2021. We chose to work with WolfSSL as it is the more popular choice amongst prior works [TLF+22, TLF+21b, TLF+21a, GW22], which makes it easier to compare reults.

[1] https://www.wolfssl.com/
[2] https://github.com/PQClean/PQClean/
[3] https://github.com/xuganyu96/embedded-pqtls
[4] https://github.com/Mbed-TLS/mbedtls

3.1.1 Integrating PQC into WolfCrypt

PQClean is a repository of thoroughly tested, high-confidence C implementations of KEM and signature schemes submitted to NIST's PQC standardization project. The "clean" implementations ensure that C programming best practices such as memory safety and integer overflows are enforced and that the correctness and basic security defense such as avoiding branching on secret data are continuously tested. The project also improved developer ergonomics by applying consistent namespacing on exported symbols and a uniform set of APIs across all curated submissions.

PQClean comes with "batteries-included": it contains reusable symmetric primitives (AES, SHA-2, and SHA-3) shared across downstream implementations. The inclusion of shared symmetric primitives made integrating PQClean into WolfCrypt easier, but unfortunately it duplicates existing functionalities from WolfCrypt, which inflates the size of the firmware. In addition, PQClean's symmetric primitives exhibit different performance characteristics from WolfCrypt: for example, WolfCrypt contains an in-house implementation of ML-KEM and ML-DSA, which uses an in-house implementation of Keccak/SHA-3. WolfCrypt's own benchmark showed that on the embedded client, WolfCrypt's SHA-3 to have approx. 10% more throughput than PQClean's `fips202.c`, while WolfCrypt's ML-KEM is roughly twice as fast as the "clean" ML-KEM. In the end, we chose to disable WolfCrypt's ML-KEM/ML-DSA and use the "clean" implementations (using PQClean's symmetric primitives) across the entire project to ensure fair comparison across different schemes. We leave it for future works to incorporate machine-optimized implementations such as from `pqm4`[5] and `ml-kem-native`[6].

While we kept PQClean's symmetric primitives, we had to modify its `randombytes` API for generating random number. PQClean's existing `randombytes` implementation is stateless and asssumes the existence of a filesystem on Linux, BSD, or Win32. While this works on a desktop environment, it does not work with baremetal embedded clients. In addition, most PQC implementations do not expose internal randomness through some `seed` parameter in the public API, so a wholesale replacement of `randombytes` by another RNG implementation will require extensive refactor of the entire project. Fortunately, WolfCrypt provides an abstraction `WC_RNG` with ready-to-go backends on both Linux and baremetal via Pico-SDK. We adapted the stateless `randombytes` API to use the stateful `WC_RNG` by adding a static `WC_RNG *` pointer in PQClean and implementing a function that sets this static pointer to a user-specified instance of `WC_RNG` (Listing 3.1). This also means

[5] https://github.com/mupq/pqm4
[6] https://github.com/pq-code-package/mlkem-native

that all PQClean schemes will source from the same randomness used throughout the TLS handshake.

Listing 3.1: Use `WC_RNG` as backend for PQClean's `randombytes` API

```
#ifdef HAVE_WC_RNG
#include <wolfssl/wolfcrypt/random.h>
static WC_RNG *g_rng;
void set_wc_rng(WC_RNG *input) {
    g_rng = input;
}
#endif

int randombytes(uint8_t *output, size_t n) {
    void *buf = (void *)output;
    #ifdef HAVE_WC_RNG
    (void)buf;
    return wc_RNG_GenerateBlock(g_rng, output, n);
    #endif
}
```

`liboqs`[7] is another repository of high-quality PQC implementations that contain a wider selection of schemes, some of which are automatically integrated from PQClean. We considered using `liboqs`, but found the lack of documentation on baremetal systems (`arm-none-eabi-gcc` toolchain) and the complexity of the project difficult to navigate. PQClean's simple structure allowed us to easily use a single codebase and build for both server (`x86_64` on Linux) and embedded client with no RTOS.

3.1.2 Implementing post-quantum key exchange

Adding new KEM Incorporating post-quantum KEMs into the key exchange flow of TLS 1.3 is relatively straightforward despite the API difference between a KEM and a Diffie-Hellman key exchange. In fact, WolfSSL already supports ML-KEM for the key exchange phase, which provides a template with which we can integrate HQC and IND-1CCA ML-KEM. RFC 8446 specified that each key exchange group (called `NamedGroup`) is

[7] https://github.com/open-quantum-safe/liboqs

identified with a 16-bit value, which is captured in an enum type `NamedGroup` in WolfSSL. The variants of this enum type are assigned their group point to be directly used in constructing the `key_share` extension the hello messages. The TLS state machine then uses a case-switch block to call the correct lower-level subroutines based on the variant. Figure 3.2 provides an example.

Listing 3.2: Use `NamedGroup` enum to direct `keygen` calls to the underlying primitive

```
static int TLSX_KeyShare_GenPqcKeyClient(WOLFSSL *ssl,
                                         KeyShareEntry* kse) {
    int ret = 0;
    switch (kse->group) {
        case WOLFSSL_ML_KEM_512:
        case WOLFSSL_ML_KEM_768:
        case WOLFSSL_ML_KEM_1024:
            ret = TLSX_KeyShare_GenMlKemKeyClient(ssl, kse);
            break;
        case HQC_128:
        case HQC_192:
        case HQC_256:
            ret = TLSX_KeyShare_GenHqcKeyClient(ssl, kse);
            break;
        case OT_ML_KEM_512:
        case OT_ML_KEM_768:
        case OT_ML_KEM_1024:
            ret = TLSX_KeyShare_GenOtMlKemKeyClient(ssl, kse);
            break;
        default:
            ret = NOT_COMPILED_IN;
            break;
    }
    return ret;
}
```

The explicit case-switch statement stands in contrast with TLS libraries implemented using languages with more sophisticated abstraction such as generics. For example, `rustls`, a TLS library written in Rust, takes advantage of Rust's traits/generics syntax, which enabled user to provide its own cryptographic backend by implementing a `CryptoProvider`

struct (Listing 3.3). While the lack of ergonomic generics in the C programming language made much of WolfSSL's codebase verbose and sometimes difficult to read long case-switch or if-else blocks, however, the explicitness also made it easy to reason about the state machine logic. We especially appreciate this flat abstraction when using a visual debugger such as `lldb`.

Listing 3.3: User can provide custom cryptographic backend to `rustls` through the `CryptoProvider` struct

```
pub struct CryptoProvider {
    pub cipher_suites: Vec<SupportedCipherSuite>,
    pub kx_groups: Vec<&'static dyn SupportedKxGroup>,
    pub signature_verification_algorithms: WebPkiSupportedAlgorithms,
    pub secure_random: &'static dyn SecureRandom,
    pub key_provider: &'static dyn KeyProvider,
}
```

Memory management Programming in C brings substantial challenge in memory management, especially on the embedded client, where memory leaks can quickly exhaust the meager amount of available RAM. Following best practices, we allocate memory for cryptographic operations on the stack wherever possible, and on the heap only when necessary, such as when data need to persist across multiple TLS messages. With the ephemeral key exchange, KEM public key and private key must be dynamically allocated because they need to live after the local frame exits. On the other hand, ciphertexts and shared secrets are short-lived and are thus allocated on the stack. Listing 3.4 provides an example of managing memories for KEM keys.

Miscellaneous One interesting design is a `static const word16 preferredGroup[]`. User can specify the key exchange groups using the `wolfSSL_CTX_set_groups` API, but this user-specified list of groups is copmared with the internal static `preferredGroups` when constructing `ClientHello`. If the two lists do not match, then WolfSSL will send a `ClientHello` with an empty `key_share` extension; per RFC 8446, this indicates that client is requesting group selection from the server, and will increase handshake latency by one round trip. We were unaware of this feature and had spent a lot of effort trying to find out why the client is sending `ClientHello` twice even though client and server both support the newly added KEM.

Listing 3.4: Managing memory for KEM key

```
static int TLSX_KeyShare_GenPQCleanMlKemKeyClient(WOLFSSL *ssl,
                                                  KeyShareEntry *kse) {
    int ret = 0;
    PQCleanMlKemKey kem;
    byte *privKey = NULL;
    word32 privKeyLen = 0;
    /* ... fill in privKeyLen, kse->pubKeyLen ... */
    if (ret == 0) {
        privKey = XMALLOC(privKeyLen, ssl->heap,
                          DYNAMIC_TYPE_PRIVATE_KEY);
        if (privKey == NULL)
            ret = MEMORY_ERROR;
    }
    if (ret == 0) {
        kse->pubKey = XMALLOC(kse->pubKeyLen, ssl->heap,
                              DYNAMIC_TYPE_PUBLIC_KEY);
        if (kse->pubKey == NULL)
            ret = MEMORY_ERROR;
    }
    /* ... generate key pair ... */
    if (ret != 0) {
        if (privKey)
            XFREE(privKey, ssl->heap, DYNAMIC_TYPE_PRIVATE_KEY);
        if (kse->pubKey) {
            XFREE(kse->pubKey, ssl->heap, DYNAMIC_TYPE_PUBLIC_KEY);
            kse->pubKey = NULL;
        }
    } else {
        kse->privKey = privKey;
        kse->privKeyLen = privKeyLen;
    }
    return ret;
}
```

Last but not least, WolfSSL included ECC+KEM hybrids for key exchange, but the integration with WolfCrypt's ML-KEM is too complex for us to modify in time. We included benchmark results from these hybrid key exchange groups in Section 4.4 but they are not directly comparable with other ML-KEM-only key exchange.

3.1.3 Generating certificate chain and private keys

Because we need to distribute related components of a public-key certificate chain to the server and the client separately, we need to generate the certificate and the private key files. X.509 is the standard format for encoding a public-key certificate. It is based on the Abstract Sytnax Notation One (ASN.1) and is usually serialized according to Distinguished Encoding Rule (DER). DER is a binary format and not human-readable. We chose to further encode the DER output into PEM, which is a human-readable format that exclusively uses ASCII characters. Although the PEM-format requires a larger size to store the same certificate/private key because it is a Base64 encoding of the DER data enclosed in header and footer, it is also much easier to work with, as we could trivially load a PEM-formatted certificate into the embedded client's firmware using C macros (Listing 3.5).

Listing 3.5: PEM-formatted certificate can be loaded without a file system

```
#define CA_CERT                     \
    "-----BEGIN CERTIFICATE-----"  \
    "..."                           \
    "-----END CERTIFICATE-----"
/* ... setting up WolfSSL ... */
uint8_t ca_certs[] = CA_CERT;
size_t ca_certs_size = sizeof(ca_certs);
wolfSSL_CTX_set_verify(ctx, WOLFSSL_VERIFY_PEER, NULL);
ssl_err = wolfSSL_CTX_load_verify_buffer(ctx, ca_certs, ca_certs_size,
                                         SSL_FILETYPE_PEM);
if (ssl_err != SSL_SUCCESS) {
    /* fail */
}
```

WolfSSL includes an ASN.1 module that can generate, sign, encode, and parse public-key certificates and private keys (Listing 3.6).

Listing 3.6: Generate, sign, then PEM-encode certificate with WolfSSL

```
int ret;
WC_RNG rng;
MlDsaKey key;
Cert cert;
uint8_t der[BUF_SIZE], pem[BUF_SIZE];
int derSz, pemSz;
/* ... initialize signature keypairs and RNG
wc_InitCert(&cert);
cert.sigType = CTC_ML_DSA_LEVEL1;
/* ... fill in subject, issuer, valid dates ... */
derSz = wc_MakeCert_ex(&cert, der, sizeof(der), ML_DSA_LEVEL2_TYPE,
                       &key, &rng);
derSz = wc_SignCert_ex(cert.bodySz, cert.sigType, der, sizeof(der),
                       ML_DSA_LEVEL2_TYPE, &key, &rng);
pemSz = wc_DerToPem(der, derSz, pem, sizeof(pem), CERT_TYPE);
```

Thanks to existing scaffolding for supporting ML-DSA, expanding WolfSSL's ASN.1 engine to support generating certificates containing ML-KEM/HQC/Falcon/SPHINCS+ public key, and to support signing certificate body with Falcon/SPHINCS+ private keys is straightforward. We particularly appreciate the clever abstraction with which WolfSSL's `MakeCert` and `SignCert` API can support more than a dozen signature scheme/variants while maintaining a compact function signature (Listing 3.7).

Last but not least, an Object Identifier (OID) is needed to identify the public key and the signature on a public-key certificate (as well as the type of key in a private key file). The OIDs for ML-KEM, ML-DSA, and SPHINCS+ (SLH-DSA) were standardized by NIST [8], so we extended them for HQC and Falcon.

3.1.4 Implementing KEMTLS

Re-write!

[8] https://csrc.nist.gov/projects/computer-security-objects-register/algorithm-registration

Listing 3.7: Using enum as hint to cast void pointers to the correct type allows `MakeCert`/`SignCert` to have compact signature despite lack of generics in C

```
int wc_MakeCert_ex(Cert* cert, byte* derBuffer, word32 derSz,
                   int keyType, void* key, WC_RNG* rng) {
    /* ... other key types ...*/
    falcon_key*        falconKey = NULL;
    dilithium_key*     dilithiumKey = NULL;
    sphincs_key*       sphincsKey = NULL;
    if (keyType == FALCON_LEVEL1_TYPE)
        falconKey = (falcon_key*)key;
    else if (keyType == FALCON_LEVEL5_TYPE)
        falconKey = (falcon_key*)key;
    else if (keyType == ML_DSA_LEVEL2_TYPE)
        dilithiumKey = (dilithium_key*)key;
    else if (keyType == ML_DSA_LEVEL3_TYPE)
        dilithiumKey = (dilithium_key*)key;
    else if (keyType == ML_DSA_LEVEL5_TYPE)
        dilithiumKey = (dilithium_key*)key;
    else if (keyType == SPHINCS_FAST_LEVEL1_TYPE)
        sphincsKey = (sphincs_key*)key;
    else if (keyType == SPHINCS_FAST_LEVEL3_TYPE)
        sphincsKey = (sphincs_key*)key;
    else if (keyType == SPHINCS_FAST_LEVEL5_TYPE)
        sphincsKey = (sphincs_key*)key;
    else if (keyType == SPHINCS_SMALL_LEVEL1_TYPE)
        sphincsKey = (sphincs_key*)key;
    else if (keyType == SPHINCS_SMALL_LEVEL3_TYPE)
        sphincsKey = (sphincs_key*)key;
    else if (keyType == SPHINCS_SMALL_LEVEL5_TYPE)
        sphincsKey = (sphincs_key*)key;
    /* ... build cert with key and encode DER to derBuffer ... */
}
```

Implementing unilaterally authenticated KEMTLS handshake

KEMTLS handshake workflow is identical to TLS 1.3's handshake flow from the beginning until client starts processing server's `Certificate` message. Even after KEMTLS and TLS 1.3's handshake flow diverges, they still share the format of the `Finished` message (which contains exactly one HMAC tag). Finally, once the handshake is complete, TLS 1.3 and KEMTLS exchange application data in identical fashion. The similarity between TLS 1.3 and KEMTLS handshake workflow allows us to reuse a significant part of WolfSSL's TLS 1.3 implementation, diverging at only a handful of places that are easy to reason about. While working with a TLS library written in C is intimidating at first, this implementation strategy proved successful, and I was able to finish implementing KEMTLS in less than a month using only around 4600 lines of code change.

The `WOLFSSL` struct is used on both client-side and server-side and encodes the pair of client and server state as a global TLS state. We begin modifying the TLS state machine by adding two flags `haveMlKemAuth` and `haveHqcAuth` to the main `WOLFSSL` struct. In a unilaterally authenticated KEMTLS handshake, the two flags are set on the server side when the server loads a KEM private key. Detecting a KEM private key is cleanly accomplished because at certificate generation, private keys are encoded according to DER, and the OID of the KEM scheme is included. If the OID belongs to one of ML-KEM's variants, then `haveMlKemAuth` is set, and if the OID belongs to one of HQC's variants, then `haveHqcAuth` is set. On the client side, these two flags are set when the client finds a ML-KEM or HQC public key in the certificate chain sent by the server. The combination of these two flags is sufficient for deciding when the two peers are performing a KEMTLS or TLS 1.3 handshake, and all divergence between KEMTLS and TLS 1.3 handshake flow will be controlled by these two flags.

On the client side, KEMTLS and TLS 1.3 handshake flows first diverge after the client finishes processing server's `Certificate` message. In signature-based TLS 1.3, client's immediate next step is to receive and process server's `CertificateVerify` containing a signature over the handshake transcript. In KEMTLS, client will not receive additional message. Instead, it uses the KEM public key to encapsulate the **authentication secret**, then sends the ciphertext to the server in a `KemCiphertext` message. We followed the original KEMTLS implementation's `KemCiphertext` format as a handshake message whose payload contains the raw ciphertext and no additional metadata. Within the context of this project, each server instance will only load one private key, so `KemCiphertext` not carrying metadata on the ciphertext will not cause confusion. However, WolfSSL supports loading multiple private keys for authentication, in which case it might be necessary for `KemCiphertext` to carry metadata such as an OID.

Client's `KemCiphertext` is encrypted under handshake traffic keys derived from the unauthenticated handshake secret (HS). After sending `KemCiphertext`, client must update the key schedule by mixing in the authentication secret and deriving the authenticated handshake secret (AHS). From AHS, the client will derive new handshake traffic key for encrypting additional handshake messages, as well as application traffic key. This key schedule update is necessary for subsequent messages to provide implicit authentication: no adversary, even if it compromises the handshake secret, can decrypt subsequent handshake message or application data without the long-term secret key.

Client's `Finished` follows the same format as TLS 1.3's `Finished`: a handshake message whose payload contains a raw HMAC tag computed under the `finished_key` against the handshake transcript. The MAC key is derived from the `MasterSecret`, which is itself derived from AHS. After sending `Finished`, the client can start sending application data without receiving server's `Finished` first. Because application data will be encrypted under application traffic key derived from authenticated handshake secret, any server successfully decrypting them is implicitly authenticated. Finally, client explicitly authenticates the server after receiving server's `Finished`.

On the server side, KEMTLS and TLS 1.3 handshake flow first diverges after server finishes sending its `Certificate`. If either of `haveMlKemAuth` and `haveHqcAuth` flag is set, then after exiting `SendTls13Certificate`, instead of constructing and sending `CertificateVerify`, server will receive and process `KemCiphertext`. After decapsulating client's `KemCiphertext`, server similarly needs to update the key schedule: first derive the authenticated handshake secret using the authentication secret, then derive new handshake traffic keys, HMAC keys for processing client's `Finished` and constructing server's `Finished`, and finally the application traffic keys. Last but not least, server needs to construct and send its `Finished` before it can start sending application data.

In our implementation, we used the handshake message type for `client_key_exchange` when constructing `KemCiphertext`. `client_key_exchange` is a handshake message type that exists only in TLS 1.2 or prior but not in TLS 1.3. This upsets some sanity checks in WolfSSL's TLS 1.3 state machine. Similarly, the different order of messages, particularly the order of `Finished`, can also cause the same set of sanity checks to fail. For the purpose of benchmarking performance only, we modified the these sanity checks to ignore message orders when `haveMlKemAuth` or `haveHqcAuth` is set. However, in production use, KEMTLS will definitely require a distinct set of checks to ensure the integrity of the handshake state machine.

3.1.5 Miscellaneous comments

We appreciate the many thoughtful design choices made in both WolfSSL and WolfCrypt. Using C preprocessing macros defined in a easily manageable `user_settings.h` file allowed us to build for three different platforms (Apple Silicon on the author's laptop running MacOS, `x86_64` on the test server running Linux, and 32-bit ARM on baremetal) using a single codebase. The modular I/O callback API made it possible to run WolfSSL on the Pi Pico without any RTOS. WolfSSL's repository even provided ready-made integration with the `Pico-SDK` so that using Pico's hardware RNG with WolfCrypt's `WC_RNG` struct requires no additional effort from the authors.

Another difficulty with WolfSSL comes from the need for manual memory management. This is especially the case when building for the Pico, which has only 512kB SRAM and can be easily overwhelmed by memory leaks since post-quantum keys, ciphertexts, and signatures all consume 1-10 kilobytes of memories each. Fortunately, there are only a handful of places where WolfSSL requires dynamic memory allocations, and they are either freed within the function scope or freed at the end of the handshake as part of `wolfSSL_shutdown` or `wolfSSL_free`. Other instances of dynamic memory allocation were flawlessly managed in WolfSSL's existing source code, allowing the Pico to continuously perform thousands of handshakes without exhausting its SRAM or needing assistance from an RTOS.

Chapter 4

Performance of post-quantum TLS 1.3 and KEMTLS

This section will present and analyze performance benchmarking data. Section 4.1 describes the hardware setup and the software toolchain. Section 4.2 analyzes the communication costs, including the sizes of the various certificate chains. Section 4.3 summarizes the performance of relevant cryptographic primitives on the test hardware. Finally Section 4.4 details the performance of post-quantum TLS 1.3 and KEMTLS.

4.1 Experiment setup

4.1.1 Client

Hardware The Raspberry Pi Pico 2 W (Figure 4.1) is a microcontroller developed by the Raspberry Pi Foundation and was released in late 2024. At the heart of the Pico 2 W is the RP2350 chip, which contains two ARM Cortex-M33 cores and two Hazard3 RISC-V cores, although only one architecture can be enabled at a time. The chip is clocked at 150MHz. The Pico 2 W also has 520KB of SRAM and 4MB of flash storage. For connectivity, the Pico 2 W has a CYW43439 module that supports 2.4 GHz Wifi and Bluetooth 5.4.

Software Toolchain For developing on the Pico 2 W, we used the GNU ARM toolchain (`arm-none-eabi-gcc` version 8.5.0) and the Pico-SDK, which comes with a standard library as well as integration with third-party open source projects, most notably `lwip`

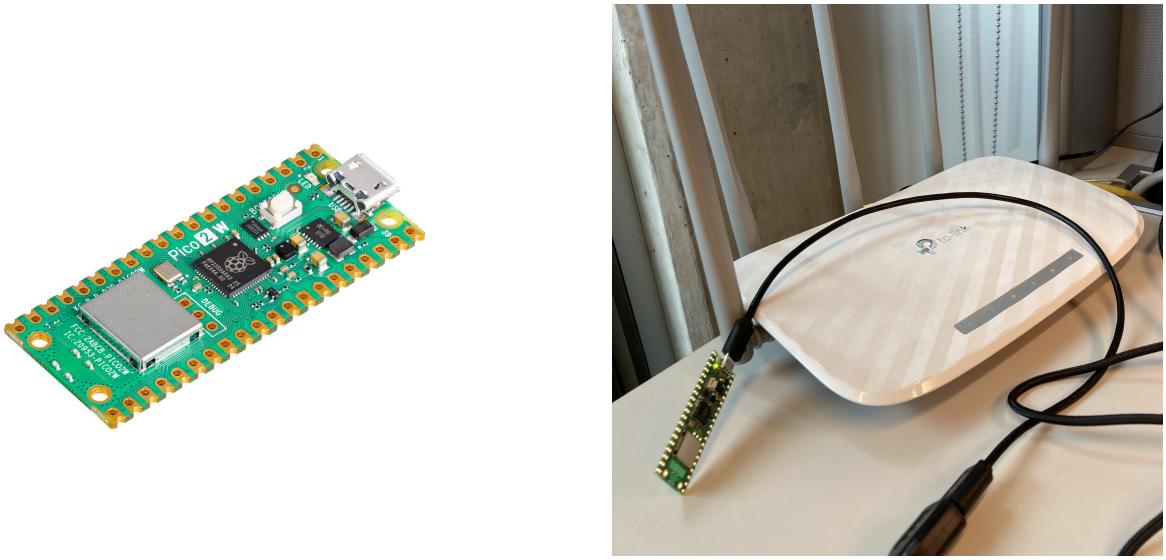

Figure 4.1: Client hardware: Pi Pico 2 W connected via 2.4GHz WiFi

(lightweight IP). `lwip` is a TCP/IP library designed for embedded system, and the Pico-SDK provided tight integration between `lwip` and the `cyw43` driver, which allowed us to implement a simple TCP stream on top of the raw TCP API. We also used `lwip` to implement DNS and time synchronization over NTP, the latter being necessary for WolfSSL to validate the expiration dates of peer's certificates. Because of the lack of a file system, certificates must be baked into the firmware at compilation. While it is possible to define certificates as raw bytes in C preprocessing macro, we found it much easier to encode the certificates in PEM format, which consists entirely of ASCII characters.

4.1.2 Server and network

Server The TLS server runs on a desktop computer with an 8-core AMD Ryzen 7 1700X clocked at 3.4GHz and 12GB of DDR4 RAM. The desktop runs Ubuntu 24.04 LTS. The server binary is compiled using `GCC 13.3.0` and `CMAKE_BUILD_TYPE=RelWithDebInfo`. Turbo boosting was disabled from the BIOS.

Network Our network setup takes advantage of Pico 2W's wireless capabilities. The Pico 2W is connected to a TP-LINK wireless access point via 2.4GHz WiFi (Figure 4.1). The access point is then connected via Ethernet to the internal network of University of

Waterloo. The server is directly connected to the same internal network via Ethernet, as well. According to `traceroute`, there is one more hop (an internal nameserver) between the access point and the server. The average network latency is less than 10 milliseconds. We choose to connect to an internal network with Internet access instead of a direct connection because we want to simulate a realistic peer verification policy that checks the expiration dates of peer's certificates. To achieve this, we adapted the example Network Time Protocol (NTP) client from Pico-SDK's repository [1] and synchronized time with `pool.ntp.org`. We also used `lwip`'s DNS API to resolve the server IP address from its hostname. Time synchronization and DNS lookup are both performed once at Pico 2 W's boot time and do not count toward TLS handshake measurements.

4.2 Communication cost

TLS 1.3 recommended ephemeral elliptic curve Diffie-Hellman key agreement (ECDHE) to be the default scheme for key exchange in `ClientHello` and `ServerHello`. Using ECDHE, the client and the server each sends the other an element from the agreed group of points on an elliptic curve, so round-trip communication cost is the sum of two group points. When using a post-quantum KEM, the client sends to the server an encapsulation key, and the server sends back a ciphertext obtained under said encapsulation key. The round-trip communication cost of a KEM key agreemnt is thus the sum of a public key and a ciphertext. Table 4.1 compares the communication costs between ECDHE and various post-quantum KEMs. Note that NIST curves (P-256, P-384, P-521) can be represented using compressed or uncompressed forms, but per RFC 8446 [Res18], ECDHE in TLS 1.3 always uses the uncompressed form, hence only uncompressed forms are listed.

Post-quantum digital signature schemes also have larger sizes than elliptic-curve signatures and RSA-signatures. The round trip cost of digital signature is the sum of public key size and signature size. Table 4.2 compares the sizes of public key and signature across RSA, elliptic-curve, and a selection of post-quantum signature schemes.

In practice, the `Certificate` message often contains a certificate chain. In our experiment setup, we generate a server certificate chain with three certificates: a leaf certificate for which the server holds the private key, an intermediate certificate authority who signs server's publickey, and a root certificate authority who signs the intermediate certificate authority's public key. However, root CA's certificate is directly compiled into the client firmware. so server's `Certificate` only contains the leaf certificate and the intermediate

[1] https://github.com/raspberrypi/pico-examples/blob/master/pico_w/wifi/ntp_client

Table 4.1: Key exchange communication costs (bytes)

Category	Scheme	Public key	Ciphertext	Round trip
ECDHE	P-256	65		130
	P-384	97		194
	P-521	133		266
	X25519	32		64
	X448	56		112
PQC	ML-KEM-512	800	768	1568
	ML-KEM-768	1184	1088	2272
	ML-KEM-1024	1568	1568	3136
	HQC-128	2249	4433	6682
	HQC-192	4522	9029	13551
	HQC-256	7245	14469	21714

Table 4.2: Digital signatures communication costs (bytes)

Category	Scheme	Public key	Signature	Total
Classical	RSA-2048 (PKCS#1 v1.5)	256	256	512
	ECDSA P-256	64	64	128
	ECDSA P-384	96	96	192
	ECDSA P-521	132	132	264
	Ed25519	32	64	96
	Ed448	57	114	171
PQC	ML-DSA-44	1312	2420	3732
	ML-DSA-65	1952	3309	5261
	ML-DSA-87	2592	4627	7219
	Falcon-512	897	666	1563
	Falcon-1024	1793	1280	3073
	SPHINCS+-128f	32	17088	17120
	SPHINCS+-192f	48	35664	35712
	SPHINCS+-256f	64	49856	49920
	SPHINCS+-128s	32	7856	7888
	SPHINCS+-192s	48	16224	16272
	SPHINCS+-256s	64	29792	29856

CA certificate. Table 4.3 compares the size of root CA and server's certificate chain size across a variety of configurations. Note that these sizes are measured with PEM encoding, whereas the `Certificate` message contains certificates encoded using DER. Since PEM is the base-64 encoding of the raw bytes of DER plus a human-readable header and footer, the sizes presented in Table 4.3 are proportionally larger than what is actually transmitted in a handshake.

Table 4.3: Server certificate chain size (bytes).

Root	**Root cert size**	**Intermediate**	**Leaf**	**Server chain size**
		Classical		
RSA-2048	1265	RSA-2048	RSA-2408	2506
ECDSA P-256	???	ECDSA P-256	ECDSA P-256	???
Ed25519	???	Ed25519	Ed25519	???
ECDSA P-384	???	ECDSA P-384	ECDSA P-384	???
Ed448	???	Ed448	Ed448	???
ECDSA P-521	???	ECDSA P-521	ECDSA P-521	???
		PQ-TLS		
ML-DSA-44	???	ML-DSA-44	ML-DSA-44	???
ML-DSA-65	???	ML-DSA-65	ML-DSA-65	???
ML-DSA-87	???	ML-DSA-87	ML-DSA-87	???
Falcon-512	???	Falcon-512	Falcon-512	???
Falcon-1024	???	Falcon-1024	Falcon-1024	???
SPHINCS-128f	???	SPHINCS-128f	SPHINCS-128f	???
SPHINCS-192f	???	SPHINCS-192f	SPHINCS-192f	???
SPHINCS-256f	???	SPHINCS-256f	SPHINCS-256f	???
SPHINCS128s	???	SPHINCS128s	SPHINCS128s	???
SPHINCS192s	???	SPHINCS192s	SPHINCS192s	???
SPHINCS256s	???	SPHINCS256s	SPHINCS256s	???
Falcon-512	???	Falcon-512	ML-DSA-44	???
Falcon-1024	???	Falcon-1024	ML-DSA-65	???
SPHINCS128s	???	SPHINCS128s	ML-DSA-44	???
		KEMTLS		
ML-DSA-44	???	ML-DSA-44	ML-KEM-512	???
Falcon-512	???	Falcon-512	ML-KEM-512	???
SPHINCS128s	???	SPHINCS128s	HQC-128	???

4.3 Cryptographic operations performance

We benchmarked the speed of relevant cryptographic operations on the Pico 2 W. We measure performance of cryptographic routines with the number of CPU cycles, which is read from the `CYCCNT` register, located at `PPB_BASE + M33_DWT_CYCCNT_OFFSET` according to Pico 2 W's data sheet. Table 4.4 and 4.5 summarizes the results.

Table 4.4: Key exchange performance on RP2350 (cycles/tick)

ECDHE						
Name	`KeyGen`			`Agree`		
	Medium	P90	P99	Medium	P90	P99
X25519						
X448						
P-256						
P-384						
P-521						
Post-quantum KEM						
	`KeyGen`			`Encap`		
	Medium	P90	P99	Medium	P90	P99
ML-KEM-512						
ML-KEM-768						
ML-KEM-1024						
OT-ML-KEM-512						
OT-ML-KEM-768						
OT-ML-KEM-1024						
HQC-128						
HQC-192						
HQC-256						

	`Decap`		
	Medium	P90	P99
ML-KEM-512			
ML-KEM-768			
ML-KEM-1024			
OT-ML-KEM-512			
OT-ML-KEM-768			
OT-ML-KEM-1024			
HQC-128			
HQC-192			
HQC-256			

Table 4.5: Digital signature performance on RP2350 (cycles/tick)

	KeyGen			Sign			Verify		
	Medium	P90	P99	Medium	P90	P99	Medium	P90	P99
Classical									
RSA-2048									
Ed25519									
ECDSA (P-256)									
ECDSA (P-384)									
ECDSA (P-521)									
Post-quantum									
ML-DSA-44									
ML-DSA-65									
ML-DSA-87									
SPHINCS+-128f									
SPHINCS+-128s									
SPHINCS+-192f									
SPHINCS+-192s									
SPHINCS+-256f									
SPHINCS+-256s									
Falcon-512									
Falcon-1024									

4.4 Handshake latency

To analyze and compare handshake latency data, we modified the WolfSSL library to collect timestamps at important steps of the handshake on the client side. The timestamps are collected using the CPU clock, which is not a real-time clock but can precisely measure time interval (within 1-2 microseconds per 150,000 microseconds). The timestamps include:

- `ch_start`: the beginning of the handshake
- `keygen_start`: when client starts generating the ephemeral keypair
- `keygen_done`: when client finishes generating the ephemeral keypair
- `ch_sent`: when client finishes sending `ClientHello`

- `sh_start`: when client start processing `ServerHello`
- `decap_start`: when client start processing server's key share in `ServerHello`. With ECDHE, this means running the `agree` sub-routine; with KEMs, this means running the `decaosulation` sub-routine
- `decap_done`: when client finishes processing server's key share, but before updating key schedule
- `sh_done`: when client finishes processing `ServerHello`, which includes updating key schedule and deriving handshake secret
- `cert_start`: when client starts processing `Certificate`
- `auth_start`: when client starts the asymmetric cryptographic operation for server authentication. In signature-based workflow, this is right before client starts verifying the signature in `CertificateVerify`. In KEM-based workflow, this is right before client starts encapsulating using server's public key.
- `auth_done`: when client finishes the asymmetric cryptographic operation for server authentication
- `hs_done`: when client finishes processing server's `Finished`. Note that in signature-based workflow, this is the earliest moment when client can start sending application data, but in KEM-based workflow, client can start at an earlier moment in the handshake. Due to time constraint we did not get to implement this "early application data", but we speculate that the performance impact will be minimal.

Although communication cost contrinutes significantly to the latency of a handshake in both the key exchange phase and the authentication phase, measuring standalone "certificate transmission time" is not straightforward as it requires high precision time synchronization between the client and the server. Unfortunately, the NTP client can have up to hundreds of milliseconds of bias and is thus rather not helpful. It is also questionable whether standalone transmission time is all that meaningful, because data transmission often happens concurrently with data processing. For example, server transmitting `Certificate` can happen concurrently with client processing `ServerHello`. Therefore, longer transmission time by itself does not necessarily translate to longer handshake latency. Instead, we will only measure the combined duration of data processing and data transmission. Finally, each configuration is repeated for at least 1,000 times.

Key exchange metrics Within the key exchange phase we mainly care about three metrics:

- *Total time*: the duration of the entire key exchange from the client's perspective, which is the interval from `ch_start` to `sh_done`.
- *CPU time*: the time spent constructing `ClientHello` and processing `ServerHello`, equivalently the time during which client's CPU is doing active work. It is computed as `ch_sent - ch_start + sh_done - sh_start`.
- *Crypto time*: the time spent on generating client's key share and processing server's key share. With ECDHE, this means running `keygen` and `agree` each once; with KEM, this means running `keygen` and `decap` each once. Crypto time should be a subset of CPU time, since the client needs to process other components of the two messages and update the key schedule to derive handshake secret. It is computed as `keygen_done - keygen_start + decap_done - decap_start`

Key exchange	**Crypto time**		**CPU time**		**Total time**	
	Median	Std	Median	Std	Median	Std
ECDHE						
P-256	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
X25519	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
PQ KEM						
ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
OT-ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
HQC-128	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ECC+KEM hybrid						
P-256 + ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
P-256 + ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
X25519 + ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
X25519 + ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx

Table 4.6: Key exchange phase duration (μs) with NIST level 1 security

Key exchange	Crypto time		CPU time		Total time	
	Median	Std	Median	Std	Median	Std
ECDHE						
P-384	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
X448	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
PQ KEM						
ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
OT-ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
HQC-192	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ECC+KEM hybrid						
P-384 + ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
P-384 + ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
X448 + ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx

Table 4.7: Key exchange phase duration (μs) with NIST level 3 security

Key exchange	Crypto time		CPU time		Total time	
	Median	Std	Median	Std	Median	Std
ECDHE						
P-521	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
PQ KEM						
ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
OT-ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
HQC-256	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ECC+KEM hybrid						
P-521 + ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx	xxxxxx

Table 4.8: Key exchange phase duration (μs) with NIST level 5 security

Authentication metrics For the authentication phase we mainly care about two metrics:

- *Crypto time*: the time spent on asymmetric cryptographic operations during the authentication phase. For signature-based authentication, this includes verifying the

signature in `Certificate` verify. For KEM-based authentication, this includes encapsulating the authentication secret. Note that in our KEMTLS implementation, the authentication secret is encapsulated eagerly while processing server's `Certificate`, and constructing `KemCiphertext` only involves copying the ciphertext from the internal state to the output buffer, so authentication's crypto time does not involve `KemCiphertext`.

- *Total time*: the duration of the entire authentication phase, which counts from `cert_start` to `hs_done`.

Certificate chain			**Crypto time**		**Total time**	
Root	Int	Leaf	Median	Std	Median	Std
		Classical				
RSA-2048	RSA-2048	RSA-2048	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ECDSA P-256	ECDSA P-256	ECDSA P-256	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Ed25519	Ed25519	Ed25519	xxxxxx	xxxxxx	xxxxxx	xxxxxx
		PQ-TLS				
ML-DSA-44	ML-DSA-44	ML-DSA-44	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ML-DSA-65	ML-DSA-65	ML-DSA-44	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-512	Falcon-512	Falcon-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-512	Falcon-512	ML-DSA-44	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS128s	SPHINCS128s	SPHINCS128s	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS128s	SPHINCS128s	ML-DSA-44	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS128s	SPHINCS128s	Falcon-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx
		KEMTLS				
ML-DSA-44	ML-DSA-44	ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ML-DSA-65	ML-DSA-65	ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-512	Falcon-512	ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS128s	SPHINCS128s	ML-KEM-512	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS128s	SPHINCS128s	HQC-128	xxxxxx	xxxxxx	xxxxxx	xxxxxx

Table 4.9: Authentication durations (μs) with NIST level 1 security

Certificate chain			Crypto time		Total time	
Root	Int	Leaf	Median	Std	Median	Std
Classical						
ECDSA P-384	ECDSA P-384	ECDSA P-384	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Ed448	Ed448	Ed448	xxxxxx	xxxxxx	xxxxxx	xxxxxx
PQ-TLS						
ML-DSA-65	ML-DSA-65	ML-DSA-65	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ML-DSA-87	ML-DSA-87	ML-DSA-65	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-1024	Falcon-1024	ML-DSA-65	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS192s	SPHINCS192s	ML-DSA-65	xxxxxx	xxxxxx	xxxxxx	xxxxxx
KEMTLS						
ML-DSA-65	ML-DSA-65	ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx
ML-DSA-87	ML-DSA-87	ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-1024	Falcon-1024	ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS192s	SPHINCS192s	ML-KEM-768	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS192s	SPHINCS192s	HQC-192	xxxxxx	xxxxxx	xxxxxx	xxxxxx

Table 4.10: Authentication durations (μs) with NIST level 3 security

Certificate chain			Crypto time		Total time	
Root	Int	Leaf	Median	Std	Median	Std
Classical						
ECDSA P-521	ECDSA P-521	ECDSA P-521	xxxxxx	xxxxxx	xxxxxx	xxxxxx
PQ-TLS						
ML-DSA-87	ML-DSA-87	ML-DSA-87	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-1024	Falcon-1024	Falcon-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-1024	Falcon-1024	ML-DSA-87	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS256s	SPHINCS256s	ML-DSA-87	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS256s	SPHINCS256s	Falcon-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx
KEMTLS						
ML-DSA-87	ML-DSA-87	ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx
Falcon-1024	Falcon-1024	ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS256s	SPHINCS256s	ML-KEM-1024	xxxxxx	xxxxxx	xxxxxx	xxxxxx
SPHINCS256s	SPHINCS256s	HQC-256	xxxxxx	xxxxxx	xxxxxx	xxxxxx

Table 4.11: Authentication durations (μs) with NIST level 5 security

KEX	Auth			Latency	
	Root	Int	Leaf	Median	Std
Classical					
P-256	RSA-2048	RSA-2048	RSA-2048	486970.0	103761.19
P-256	ECDSA P-256	ECDSA P-256	ECDSA P-256	810259.5	21448.88
X25519	Ed25519	Ed25519	Ed25519	xxxxxx	xxxxxx
P-384	ECDSA P-384	ECDSA P-384	ECDSA P-384	xxxxxx	xxxxxx
X448	Ed448	Ed448	Ed448	xxxxxx	xxxxxx
P-521	ECDSA P-521	ECDSA P-521	ECDSA P-521	xxxxxx	xxxxxx
PQ-TLS					
ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-DSA-44	164019.5	7742.55
ML-KEM-512	Falcon-512	Falcon-512	ML-DSA-44	xxxxxx	xxxxxx
ML-KEM-512	Falcon-512	Falcon-512	Falcon-512	xxxxxx	xxxxxx
ML-KEM-512	SPHINCS128s	SPHINCS128s	ML-DSA-44	xxxxxx	xxxxxx
ML-KEM-512	SPHINCS128s	SPHINCS128s	Falcon-512	xxxxxx	xxxxxx
OT-ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-DSA-44	xxxxxx	xxxxxx
P-256 + ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-DSA-44	xxxxxx	xxxxxx
X25519 + ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-DSA-44	xxxxxx	xxxxxx
P-256 + ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-44	xxxxxx	xxxxxx
X25519 + ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-44	xxxxxx	xxxxxx
HQC-128	SPHINCS128s	SPHINCS128s	SPHINCS128s	2540080.0	128924.41
ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-65	xxxxxx	xxxxxx
ML-KEM-768	Falcon-1024	Falcon-1024	ML-DSA-65	xxxxxx	xxxxxx
ML-KEM-768	SPHINCS192s	SPHINCS192s	ML-DSA-65	xxxxxx	xxxxxx
OT-ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-65	xxxxxx	xxxxxx
P-384 + ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-65	xxxxxx	xxxxxx
X448 + ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-DSA-65	xxxxxx	xxxxxx
P-384 + ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-DSA-65	xxxxxx	xxxxxx
HQC-192	SPHINCS192s	SPHINCS192s	ML-DSA-65	5299501.0	304464.20
ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-DSA-87	xxxxxx	xxxxxx
ML-KEM-1024	Falcon-1024	Falcon-1024	Falcon-1024	xxxxxx	xxxxxx
ML-KEM-1024	Falcon-1024	Falcon-1024	ML-DSA-87	xxxxxx	xxxxxx
ML-KEM-1024	SPHINCS256s	SPHINCS256s	ML-DSA-87	xxxxxx	xxxxxx
ML-KEM-1024	SPHINCS256s	SPHINCS256s	Falcon-1024	xxxxxx	xxxxxx
OT-ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-DSA-87	xxxxxx	xxxxxx
P-521 + ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-DSA-87	xxxxxx	xxxxxx
HQC-256	SPHINCS256s	SPHINCS256s	ML-DSA-87	9243921.0	40152.74
KEMTLS					
ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-KEM-512	189151.5	20774.16
ML-KEM-512	ML-DSA-65	ML-DSA-65	ML-KEM-512	xxxxxx	xxxxxx
ML-KEM-512	Falcon-512	Falcon-512	ML-KEM-512	xxxxxx	xxxxxx
ML-KEM-512	SPHINCS128s	SPHINCS128s	ML-KEM-512	xxxxxx	xxxxxx
HQC-128	SPHINCS128s	SPHINCS128s	HQC-128	2918399.0	223598.25
ML-KEM-768	ML-DSA-65	ML-DSA-65	ML-KEM-768	xxxxxx	xxxxxx
ML-KEM-768	ML-DSA-87	ML-DSA-87	ML-KEM-768	326002.0	35235.57
ML-KEM-768	Falcon-1024	Falcon-1024	ML-KEM-768	183946.5	16618.27
ML-KEM-768	SPHINCS192s	SPHINCS192s	ML-KEM-768	1277371.5	51458.02
HQC-192	SPHINCS192s	SPHINCS192s	HQC-192	7304369.0	243299.18
ML-KEM-1024	ML-DSA-87	ML-DSA-87	ML-KEM-1024	354981.0	35428.57
ML-KEM-1024	Falcon-1024	Falcon-1024	ML-KEM-1024	210244.5	30846.48
ML-KEM-1024	SPHINCS256s	SPHINCS256s	ML-KEM-1024	1899104.0	226068.66
HQC-256	SPHINCS256s	SPHINCS256s	HQC-256	12987320.5	445509.98

Table 4.12: Handshake latency (μs)

4.4.1 Discussion

IND-1CCA KEM We expected meaningful performance improvements from removing *re-encryption* from an FO-protected IND-CCA secure KEM. Prior implementations on SIKE/Kyber/Lightsaber [HV22] and ML-KEM [ZZD+24] reported decapsulation speedup ranging from 2x to 6x. However, the performance gains in TLS 1.3 were minimal in environments where server and clients both had desktop CPUs. On the other hand, with a constrained TLS client, client-side cryptographic operations pose a more severe bottleneck, so the speed improvements are more pronounced. Another consideration unique to an embedded client is energy consumption: as reported in [TDF+23a, SLRW21], the choice of cryptographic primitives can greatly affect energy consumption, and thus the lifetime of a battery-operated embedded device.

Figure 4.2 compares the distribution of key exchange latency and the handshake latecy between ML-KEM (IND-CCA secure) and one-time ML-KEM (IND-1CCA secure).

Figure 4.2: Comparing IND-CCA and IND-1CCA KEMs

ML-DSA vs. Falcon ML-DSA and Falcon are both lattice-based digital signature schemes selected by NIST for standardization (ML-DSA was standardized in August 2024; Falcon was selected but still in the process of being standardized). However, there are notable differences between their designs and performance characteristics that make one suitable than the other in certain scenarios. Falcon has smaller signature size (752 vs 2420 bytes at NIST level 1, 1462 vs 4627 at level 5) and smaller public key size (897 vs 1312 at level 1, 1793 vs 2592 at level 5). According to eBACS [BL25], on a Cortex-A72 (Raspberry Pi 4B)[2], Falcon has significantly faster signing and verification than ML-DSA. **Does my data support this?**. Because ML-DSA signs using the "Fiat-Shamir with abort" [Lyu09] strategy, signing time can have large variances depending on the input randomness, while Falcon's signing routine does not have this issue. On the other hand, Falcon's signing routine uses a discrete Gaussian sampler, which requires constant-time floating point arithmetics with at least 53-bits of precision. The float-point arithmetics requirement is not only difficult to fulfill, such as on legacy CPUs and/or embedded devices, but also difficult to implement without risk of introducing side-channel vulnerabilities [**?**].

While each signature scheme can make a valid certificate chain, given the performance and security characteristics mentioned above, we recommend using Falcon only for offline

[2] https://bench.cr.yp.to/results-sign/aarch64-pi4b.html

signatures (signatures are produced ahead of the handshake) while using ML-DSA for online signature (signatures are produced during handshake). For example, root and intermediate CAs can issue certificates using the Falcon, and server signs `CertificateVerify` with ML-DSA. Figure 4.3 compares the performance across various Falcon/ML-DSA certificate chain configurations.

Figure 4.3: Comparing ML-DSA/Falcon mixture chains

Using SPHINCS+ Among all post-quantum signature schemes, SPHINCS+ represent the most conservative security consideration. However, the conservative security approach comes at the cost of immense performance penalty: across all variants SPHINCS+'s signature size and signing speed are one or more magnitudes worse than its competitors. This makes SPHINCS+ unsuitable as an online signature for TLS 1.3 handshake: the conservative security margins are not useful due to the relatively short lifespan (usually no more than a year) of leaf certificates, while the slow signing and large signature size will incur unacceptable CPU and bandwidth costs on the server side. In fact, amongst all SPHINCS+ variants, only SPHINCS-128s's signature can fit into a single TLS 1.3 record-layer fragment[3]; all other variant's signatures will need to be fragmented across two or more records.

We may still consider SPHINCS+ for offline signatures. In these use cases, the "small" variants offer both smaller signatures and faster verification than the "fast" variants. Figure 4.4 compares the performance of various configurations using SPHINCS+ in the certificate chain.

Figure 4.4: Comparing SPHINCS+ chains

PQ-TLS vs KEMTLS In our setup, KEMTLS and PQ-TLS showed comparable performance with respect to handshake latency. As shown in Figure 4.5, encapsulating an authentication secret is not any faster than verifying the signature in `CertificateVerify`. The communication cost advantage of KEMTLS is also partially mitigated by the fact that in a multi-core processor such as the RP2350 in our client device, receiving `Certificate` can happen concurrently with processing `ServerHello`. We speculate that our client can receive `Certificate` much faster than it can decapsulate the ciphertext in `ServerHello`,

[3]TLS 1.3 record fragment size cannot exceed $2^{14} - 1$ bytes

hence a shorter certificate chain ended up offering no meaningful performance advantage. Other implementations [CFS+21, GW22] simulated congested or narrow-band networks, which demonstrated more sizeable performance advantage to KEMTLS.

Figure 4.5: Comparing KEMTLS with PQ-TLS

Chapter 5

Conclusion and future works

This chapter summarizes previous chapters and list some future directions to be considered.

5.1 Summary of thesis

In this thesis, we implemented post-quantum TLS and KEMTLS, then evaluated their performance characteristics on a constrained client. Chapter **??** reviewed the TLS 1.3 protocol, its security properties, and its transition to post-quantum cryptography. Chapter **??** introduced two optimizations for post-quatnum TLS 1.3: using IND-1CCA KEM for ephemeral key exchange and using KEM for server authentication. In Chapter **??** and **??**, we described in details how we incorporated post-quantum primitive into TLS 1.3 for an embedded client, then discussed the performance characteristics of various configurations.

Our main contribuion **is what**.

Limitations No optimized impls, missing SPHINCs-128f due to no fragmented `CertificateVerify`, no client authentication, missing server-side metrics.

5.2 Future works

Broaden PQC primitive support At the time of this writing, NIST's PQC standardization project is calling for a second batch of digital signature schemes that may be of interest for evaluation. We may also wish to incorporate KEMs that are not selected for NIST standardization, such as NTRU and FrodoKEM.

Explore other protocol and/or variations This work primarily focused on post-quantum TLS 1.3 and KEMTLS with unilateral authentication, but they are not the only protocols suitable for embedded systems. Future works may explore further variations on these protocols or other protocols, including but not limited to

- *Datagram TLS*: DTLS is a variation on TLS that assumes the network to be lossy (i.e. using UDP instead of TCP), for which the increased communication cost of post-quantum primitives may affect performance.
- *TLS with cached-certificates*: RFC 7924 proposed an extension to `ClientHello` called `cached_info`, through which a client can declare posession of server's certificate, thus saving the need to transmit the certificate chain; this can favor signature schemes with large public key but small signature. The equivalent KEMTLS-PDK [SSW21] may also favor KEM with large public key but small ciphertext, such as Classic McEliece.
- *QUIC* is an alternative to TLS/DTLS that offered a more modern design and superior performnace. We wish to explore how incorporating post-quantum primitives might affect the behavior of QUIC on embedded devices.

Extended performance profiling So far this thesis primarily focused on handshake latency as the sole performance metric. Future works may include a more comprehensive collection of metrics, especially on the embedded client, such as code size, memory usage, and energy consumption.

IoT security and robustness IoT devices operating in hostile environments may need to adopt defense against physical attacks, such as power analysis and fault injections. In this work, we only incorporated reference implementations with certain performance optimization, and future works we may wish to evaluate hardened/masked implementations.

References

[ACZ18] Dorian Amiet, Andreas Curiger, and Paul Zbinden. Fpga-based accelerator for post-quantum signature scheme SPHINCS-256. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):18–39, 2018.

[ADK+22] Nimrod Aviram, Benjamin Dowling, Ilan Komargodski, Kenneth G. Paterson, Eyal Ronen, and Eylon Yogev. Practical (post-quantum) key combiners from one-wayness and applications to TLS. *IACR Cryptol. ePrint Arch.*, page 65, 2022.

[AHH+19] Martin R. Albrecht, Christian Hanser, Andrea Höller, Thomas Pöppelmann, Fernando Virdia, and Andreas Wallner. Implementing rlwe-based schemes using an RSA co-processor. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(1):169–208, 2019.

[AMOW23] Nouri Alnahawi, Johannes Müller, Jan Oupický, and Alexander Wiesmaier. Sok: Post-quantum TLS handshake. *IACR Cryptol. ePrint Arch.*, page 1873, 2023.

[AMOW24] Nouri Alnahawi, Johannes Müller, Jan Oupický, and Alexander Wiesmaier. A comprehensive survey on post-quantum TLS. *IACR Commun. Cryptol.*, 1(2):6, 2024.

[BBCT21] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. OpenSSLNTRU: Faster post-quantum TLS key exchange. *CoRR*, abs/2106.08759, 2021.

[BBCT22] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. OpenSSLNTRU: Faster post-quantum TLS key exchange. In Kevin R. B. Butler and Kurt Thomas, editors, *31st USENIX Security Symposium,*

USENIX Security 2022, Boston, MA, USA, August 10-12, 2022, pages 845–862. USENIX Association, 2022.

[BBP+22] Jon Barton, William J. Buchanan, Nikolaos Pitropakis, Sarwar Sayeed, and Will Abramson. Post quantum cryptography analysis of TLS tunneling on a constrained device. In Paolo Mori, Gabriele Lenzini, and Steven Furnell, editors, *Proceedings of the 8th International Conference on Information Systems Security and Privacy, ICISSP 2022, Online Streaming, February 9-11, 2022*, pages 551–561. SCITEPRESS, 2022.

[BC20] Utsav Banerjee and Anantha P. Chandrakasan. Efficient post-quantum TLS handshakes using identity-based key exchange from lattices. In *2020 IEEE International Conference on Communications, ICC 2020, Dublin, Ireland, June 7-11, 2020*, pages 1–6. IEEE, 2020.

[BCNS14] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. *IACR Cryptol. ePrint Arch.*, page 599, 2014.

[BCNS15] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*, pages 553–570. IEEE Computer Society, 2015.

[BDC20] Utsav Banerjee, Siddharth Das, and Anantha P. Chandrakasan. Accelerating post-quantum cryptography using an energy-efficient TLS crypto-processor. In *IEEE International Symposium on Circuits and Systems, ISCAS 2020, Sevilla, Spain, October 10-21, 2020*, pages 1–5. IEEE, 2020.

[Ber21] Daniel J. Bernstein. FO derandomization sometimes damages security. Cryptology ePrint Archive, Paper 2021/912, 2021.

[BHT97] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. *SIGACT News*, 28(2):14–19, 1997.

[BJ24] Bruno Blanchet and Charlie Jacomme. Post-quantum sound cryptoverif and verification of hybrid TLS and SSH key-exchanges. In *37th IEEE Computer Security Foundations Symposium, CSF 2024, Enschede, Netherlands, July 8-12, 2024*, pages 543–556. IEEE, 2024.

[BKNS20a] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems. *IACR Cryptol. ePrint Arch.*, page 308, 2020.

[BKNS20b] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems: Integrating and evaluating kyber and SPHINCS+ with mbed TLS. In Hung-Min Sun, Shiuh-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security, Taipei, Taiwan, October 5-9, 2020*, pages 841–852. ACM, 2020.

[BL25] Daniel J. Bernstein and Tanja Lange. eBACS: ECRYPT Benchmarking of Cryptographic Systems. https://bench.cr.yp.to, 2025. Accessed: 26 June 2025.

[BR93] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249. Springer, 1993.

[CCC+23] Fabio Campos, Jorge Chávez-Saab, Jesús-Javier Chi-Domínguez, Michael Meyer, Krijn Reijnders, Francisco Rodríguez-Henríquez, Peter Schwabe, and Thom Wiggers. On the practicality of post-quantum TLS using large-parameter CSIDH. *IACR Cryptol. ePrint Arch.*, page 793, 2023.

[CFS+21] Sofía Celi, Armando Faz-Hernández, Nick Sullivan, Goutam Tamvada, Luke Valenta, Thom Wiggers, Bas Westerbaan, and Christopher A. Wood. Implementing and measuring KEMTLS. In Patrick Longa and Carla Ràfols, editors, *Progress in Cryptology - LATINCRYPT 2021 - 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6-8, 2021, Proceedings*, volume 12912 of *Lecture Notes in Computer Science*, pages 88–107. Springer, 2021.

[CHH+07] Ronald Cramer, Goichiro Hanaoka, Dennis Hofheinz, Hideki Imai, Eike Kiltz, Rafael Pass, Abhi Shelat, and Vinod Vaikuntanathan. Bounded CCA2-secure encryption. In Kaoru Kurosawa, editor, *Advances in Cryptology - ASIACRYPT 2007, 13th International Conference on the Theory and Application of Cryptology and Information Security, Kuching, Malaysia, December 2-6,*

2007, Proceedings, volume 4833 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2007.

[CHSW22] Sofía Celi, Jonathan Hoyland, Douglas Stebila, and Thom Wiggers. A tale of two models: Formal verification of KEMTLS via tamarin. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III*, volume 13556 of *Lecture Notes in Computer Science*, pages 63–83. Springer, 2022.

[CPS19] Eric Crockett, Christian Paquin, and Douglas Stebila. Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. *IACR Cryptol. ePrint Arch.*, page 858, 2019.

[Den03] Alexander W. Dent. A designer's guide to kems. In Kenneth G. Paterson, editor, *Cryptography and Coding, 9th IMA International Conference, Cirencester, UK, December 16-18, 2003, Proceedings*, volume 2898 of *Lecture Notes in Computer Science*, pages 133–151. Springer, 2003.

[DFGS15] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol candidates. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 1197–1210. ACM, 2015.

[DFGS16] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 draft-10 full and pre-shared key handshake protocol. *IACR Cryptol. ePrint Arch.*, page 81, 2016.

[DFGS21] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. *J. Cryptol.*, 34(4):37, 2021.

[DG22] Ronny Döring and Marc Geitz. Post-quantum cryptography in use: Empirical analysis of the TLS handshake performance. In *2022 IEEE/IFIP Network Operations and Management Symposium, NOMS 2022, Budapest, Hungary, April 25-29, 2022*, pages 1–5. IEEE, 2022.

[DR08] Tim Dierks and Eric Rescorla. The transport layer security (TLS) protocol version 1.2. *RFC*, 5246:1–104, 2008.

[FG14] Marc Fischlin and Felix Günther. Multi-stage key exchange and the case of google's QUIC protocol. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*, pages 1193–1204. ACM, 2014.

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554. Springer, 1999.

[FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *J. Cryptol.*, 26(1):80–101, 2013.

[GAO+23] Carlos Rubio Garcia, Abraham Cano Aguilera, Juan Jose Vegas Olmos, Idelfonso Tafur Monroy, and Simon Rommel. Quantum-resistant TLS 1.3: A hybrid solution combining classical, quantum and post-quantum cryptography. In *28th IEEE International Workshop on Computer Aided Modeling and Design of Communication Links and Networks , CAMAD 2023, Edinburgh, United Kingdom, November 6-8, 2023*, pages 246–251. IEEE, 2023.

[GDL+17] Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *IACR Cryptol. ePrint Arch.*, page 1192, 2017.

[GDL+18] Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *Int. J. Netw. Secur.*, 20(5):923–930, 2018.

[GdNC+23] Alexandre Augusto Giron, João Pedro Adami do Nascimento, Ricardo Custódio, Lucas Pandolfo Perin, and Víctor Mateu. Post-quantum hybrid KEMTLS performance in simulated and real network environments. In Abdelrahaman Aly and Mehdi Tibouchi, editors, *Progress in Cryptology - LATINCRYPT 2023 - 8th International Conference on Cryptology and Information Security in Latin America, LATINCRYPT 2023, Quito, Ecuador, October 3-6, 2023, Proceedings*, volume 14168 of *Lecture Notes in Computer Science*, pages 293–312. Springer, 2023.

[GLD+17] Xinwei Gao, Lin Li, Jintai Ding, Jiqiang Liu, R. V. Saraswathy, and Zhe Liu. Fast discretized gaussian sampling and post-quantum TLS ciphersuite. In Joseph K. Liu and Pierangela Samarati, editors, *Information Security Practice and Experience - 13th International Conference, ISPEC 2017, Melbourne, VIC, Australia, December 13-15, 2017, Proceedings*, volume 10701 of *Lecture Notes in Computer Science*, pages 551–565. Springer, 2017.

[GM82] Shafi Goldwasser and Silvio Micali. Probabilistic encryption and how to play mental poker keeping secret all partial information. In Harry R. Lewis, Barbara B. Simons, Walter A. Burkhard, and Lawrence H. Landweber, editors, *Proceedings of the 14th Annual ACM Symposium on Theory of Computing, May 5-7, 1982, San Francisco, California, USA*, pages 365–377. ACM, 1982.

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In Gary L. Miller, editor, *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.

[GRTW22] Felix Günther, Simon Rastikian, Patrick Towa, and Thom Wiggers. KEMTLS with delayed forward identity protection in (almost) a single round trip. In Giuseppe Ateniese and Daniele Venturi, editors, *Applied Cryptography and Network Security - 20th International Conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings*, volume 13269 of *Lecture Notes in Computer Science*, pages 253–272. Springer, 2022.

[GW22] Ruben Gonzalez and Thom Wiggers. KEMTLS vs. post-quantum TLS: performance on embedded systems. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *Security, Privacy, and Applied Cryptography Engineering - 12th International Conference, SPACE 2022, Jaipur, India, December 9-12, 2022, Proceedings*, volume 13783 of *Lecture Notes in Computer Science*, pages 99–117. Springer, 2022.

[HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371. Springer, 2017.

[HOKG18] James Howe, Tobias Oder, Markus Krausz, and Tim Güneysu. Standard lattice-based key encapsulation on embedded devices. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):372–393, 2018.

[HPV+24] Yacoub Hanna, Diana Pineda, Maryna Veksler, Manish Paudel, Kemal Akkaya, Mila Anastasova, and Reza Azarderakhsh. Integrating post-quantum TLS into the control plane of 5g networks. In *IEEE International Performance, Computing, and Communications Conference, IPCCC 2024, Orlando, FL, USA, November 22-24, 2024*, pages 1–8. IEEE, 2024.

[HV22] Loïs Huguenin-Dumittan and Serge Vaudenay. On ind-qcca security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 613–642. Springer, 2022.

[KAK18] Brian Koziel, Reza Azarderakhsh, and Mehran Mozaffari Kermani. A high-performance and scalable hardware architecture for isogeny-based cryptography. *IEEE Trans. Computers*, 67(11):1594–1609, 2018.

[KC24] Panos Kampanakis and Will Childs-Klein. The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections. *IACR Cryptol. ePrint Arch.*, page 176, 2024.

[KE10] Hugo Krawczyk and Pasi Eronen. Hmac-based extract-and-expand key derivation function (HKDF). *RFC*, 5869:1–14, 2010.

[KK22] Panos Kampanakis and Michael G. Kallitsis. Faster post-quantum TLS handshakes without intermediate CA certificates. In Shlomi Dolev, Jonathan Katz, and Amnon Meisels, editors, *Cyber Security, Cryptology, and Machine Learning - 6th International Symposium, CSCML 2022, Be'er Sheva, Israel, June 30 - July 1, 2022, Proceedings*, volume 13301 of *Lecture Notes in Computer Science*, pages 337–355. Springer, 2022.

[Kra10] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010.*

Proceedings, volume 6223 of *Lecture Notes in Computer Science*, pages 631–648. Springer, 2010.

[KRSS19] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, page 844, 2019.

[KSSW22] Matthias J. Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving software quality in cryptography standardization projects. In *IEEE European Symposium on Security and Privacy, EuroS&P 2022 - Workshops, Genoa, Italy, June 6-10, 2022*, pages 19–30. IEEE, 2022.

[KW16] Hugo Krawczyk and Hoeteck Wee. The OPTLS protocol and TLS 1.3. In *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*, pages 81–96. IEEE, 2016.

[LAM79] L LAMPORT. Constructing digital signatures from a one-way function. *Report SRI Intl. CSL 98*, 1979.

[Lyu09] Vadim Lyubashevsky. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *Advances in Cryptology - ASIACRYPT 2009, 15th International Conference on the Theory and Application of Cryptology and Information Security, Tokyo, Japan, December 6-10, 2009. Proceedings*, volume 5912 of *Lecture Notes in Computer Science*, pages 598–616. Springer, 2009.

[MGS23] Callum McLoughlin, Clémentine Gritti, and Juliet Samandari. Full post-quantum datagram TLS handshake in the internet of things. In Said El Hajji, Sihem Mesnager, and El Mamoun Souidi, editors, *Codes, Cryptology and Information Security - 4th International Conference, C2SI 2023, Rabat, Morocco, May 29-31, 2023, Proceedings*, volume 13874 of *Lecture Notes in Computer Science*, pages 57–76. Springer, 2023.

[MS22] Dominik Marchsreiter and Johanna Sepúlveda. Hybrid post-quantum enhanced TLS 1.3 on embedded devices. In *25th Euromicro Conference on Digital System Design, DSD 2022, Maspalomas, Spain, August 31 - Sept. 2, 2022*, pages 905–912. IEEE, 2022.

[MSCB13] Simon Meier, Benedikt Schmidt, Cas Cremers, and David A. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In Natasha

Sharygina and Helmut Veith, editors, *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.

[MWM23a] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS → post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. In *IEEE European Symposium on Security and Privacy, EuroS&P 2023 - Workshops, Delft, Netherlands, July 3-7, 2023*, pages 526–538. IEEE, 2023.

[MWM23b] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS â†' post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. *IACR Cryptol. ePrint Arch.*, page 734, 2023.

[Nat17] National Institute of Standards and Technology. Security (evaluation criteria) — post-quantum cryptography standardization, 2017. Accessed: 2025-06-24.

[PKLN21] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. *IACR Cryptol. ePrint Arch.*, page 1447, 2021.

[PKLN22] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June 2022*, pages 727–740. ACM, 2022.

[PSS21] Sebastian Paul, Felix Schick, and Jan Seedorf. Tpm-based post-quantum cryptography: A case study on quantum-resistant and mutually authenticated TLS for iot environments. In Delphine Reinhardt and Tilo Müller, editors, *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*, pages 3:1–3:10. ACM, 2021.

[PST19] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. *IACR Cryptol. ePrint Arch.*, page 1447, 2019.

[PST20] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto*

2020, Paris, France, April 15-17, 2020, Proceedings, volume 12100 of *Lecture Notes in Computer Science*, pages 72–91. Springer, 2020.

[Res18] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. *RFC*, 8446:1–160, 2018.

[SC23a] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. In Paul A. Pavlou, Vishal Midha, Animesh Animesh, Traci A. Carte, Alexandre R. Graeml, and Alanah Mitchell, editors, *29th Americas Conference on Information Systems, AMCIS 2023, Panama City, Panama, August 10-12, 2023*. Association for Information Systems, 2023.

[SC23b] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. *CoRR*, abs/2309.03338, 2023.

[Sco23] Michael Scott. On TLS for the internet of things, in a post quantum world. *IACR Cryptol. ePrint Arch.*, page 95, 2023.

[Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, pages 124–134. IEEE Computer Society, 1994.

[Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26(5):1484–1509, 1997.

[SHOD22a] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. In Giuseppe Bianchi and Alessandro Mei, editors, *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2022, Roma, Italy, December 6-9, 2022*, pages 35–42. ACM, 2022.

[SHOD22b] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. *IACR Cryptol. ePrint Arch.*, page 1556, 2022.

[SKD20a] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In Dongsu Han and Anja Feldmann, editors, *CoNEXT '20: The 16th International Conference on emerging Networking EXperiments and Technologies, Barcelona, Spain, December, 2020*, pages 149–156. ACM, 2020.

[SKD20b] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.

[SKD20c] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. *IACR Cryptol. ePrint Arch.*, page 71, 2020.

[SLRW21] Maximilian Schöffel, Frederik Lauer, Carl Christian Rheinländer, and Norbert Wehn. On the energy costs of post-quantum kems in tls-based low-power secure iot. In *IoTDI '21: International Conference on Internet-of-Things Design and Implementation, Virtual Event / Charlottesville, VA, USA, May 18-21, 2021*, pages 158–168. ACM, 2021.

[SSW20] Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1461–1480. ACM, 2020.

[SSW21] Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. In Elisa Bertino, Haya Schulmann, and Michael Waidner, editors, *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part I*, volume 12972 of *Lecture Notes in Computer Science*, pages 3–22. Springer, 2021.

[SWH+23] Markus Sosnowski, Florian Wiedner, Eric Hauser, Lion Steger, Dimitrios Schoinianakis, Sebastian Gallenmüller, and Georg Carle. The performance of post-quantum TLS 1.3. In Dario Rossi, Stefano Secci, Olivier Bonaventure, and Lili Qiu, editors, *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2023 (Short Papers), Paris, France, December 5-8, 2023*, pages 19–27. ACM, 2023.

[TDEO22] Duong Dinh Tran, Canh Minh Do, Santiago Escobar, and Kazuhiro Ogata. Hybrid post-quantum TLS formal specification in maude-npa - toward its security analysis. In Sedat Akleylek, Santiago Escobar, Kazuhiro Ogata, and Ayoub Otmani, editors, *Proceedings of the International Workshop on Formal Analysis and Verification of Post-Quantum Cryptographic Protocols colocated with the 23rd International Conference on Formal Engineering Methods (ICFEM 2022), Madrid, Spain, October 24, 2022*, volume 3280 of *CEUR Workshop Proceedings*, pages 50–64. CEUR-WS.org, 2022.

[TDF+23a] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. In Andrea Bartolini, Kristian F. D. Rietveld, Catherine D. Schuman, and Jose Moreira, editors, *Proceedings of the 20th ACM International Conference on Computing Frontiers, CF 2023, Bologna, Italy, May 9-11, 2023*, pages 366–374. ACM, 2023.

[TDF+23b] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. *IACR Cryptol. ePrint Arch.*, page 506, 2023.

[TLF+21a] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on embedded systems. *IACR Cryptol. ePrint Arch.*, page 1553, 2021.

[TLF+21b] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. Cryptology ePrint Archive, Paper 2021/1553, 2021.

[TLF+22] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. In Chunhua Su, Dimitris Gritzalis, and Vincenzo Piuri, editors, *Information Security Practice and Experience - 17th International Conference, ISPEC 2022, Taipei, Taiwan, November 23-25, 2022, Proceedings*, volume 13620 of *Lecture Notes in Computer Science*, pages 432–451. Springer, 2022.

[Wes24] Bas Westerbaan. The state of the post-quantum internet, Mar 2024.

[ZJZ24] Biming Zhou, Haodong Jiang, and Yunlei Zhao. CPA-secure KEMs are also sufficient for post-quantum TLS 1.3. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, volume 15486 of *Lecture Notes in Computer Science*, pages 433–464. Springer, 2024.

[ZZD+24] Jieyu Zheng, Haoliang Zhu, Yifan Dong, Zhenyu Song, Zhenhao Zhang, Yafang Yang, and Yunlei Zhao. Faster post-quantum TLS 1.3 based on ML-KEM: implementation and assessment. In Joaquín García-Alfaro, Rafal Kozik, Michal Choras, and Sokratis K. Katsikas, editors, *Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part II*, volume 14983 of *Lecture Notes in Computer Science*, pages 123–143. Springer, 2024.

APPENDICES

Appendix A

Cryptography definitions

A.1 Public-key encryption scheme

Syntax A public-key encryption scheme (PKE) is a collection of three routines $(\texttt{KeyGen}, \texttt{Enc}, \texttt{Dec})$ defined over some plaintext space $\mathcal{M}$ and some ciphertext space $\mathcal{C}$. Key generation $(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$ is a randomized routine that returns a keypair consisting of a public encryption key and a secret decryption key. The encryption routine $\texttt{Enc} : (\texttt{pk}, m) \mapsto c$ encrypts the input plaintext m under the input public key $\texttt{pk}$ and produces a ciphertext c. The decryption routine $\texttt{Dec} : (\texttt{sk}, c) \mapsto m$ decrypts the input ciphertext c under the input secret key and produces the corresponding plaintext. Where the encryption routine is randomized, we denote the randomness by a coin $r \in \mathcal{R}$ where $\mathcal{R}$ is called the coin space. Decryption routines are assumed to always be deterministic.

Correctness A PKE is δ-correct if

$$E\left[\max_{m \in \mathcal{M}} P\left[\texttt{Dec}(\texttt{sk}, c) \neq m \mid c \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m)\right]\right] \leq \delta$$

Where the expectation is taken with respect to the probability distribution of all possible keypairs. For many lattice-based cryptosystems, decryption failures could leak information about the secret key, although the probability of a decryption failure is low enough that classical adversaries cannot exploit decryption failure more than they can defeat the underlying lattice problems.

Security The security of PKE's is conventionally discussed using adversarial games played between a challenger and an adversary [GM82]. In the OW-ATK game (Figure A.1), the challenger samples a random keypair and a random encryption. The adversary is given the public key, the random encryption (also called the challenge ciphertext), and access to ATK, then asked to decrypt the challenge ciphertext.

OW-ATK game

$(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$
$m^* \xleftarrow{\$} \mathcal{M}$
$c^* \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m)$
$\hat{m} \xleftarrow{\$} A^{\text{ATK}}(1^\lambda, \texttt{pk}, c^*)$
return $[\![\hat{m} = m^*]\!]$

Figure A.1: The one-wayness game: challenger samples a random keypair and a random encryption, and the adversary wins if it correctly produces the decryption

The advantage of an adversary is its probability of producing the correct decryption: $\text{Adv}^{\text{OW-ATK}}_{\texttt{PKE}}(A) = P[\hat{m} = m^*]$. A PKE is said to be OW-ATK secure if no efficient adversary can win the OW-ATK game with non-negligible probability.

IND-ATK game

$(\texttt{pk}, \texttt{sk}) \xleftarrow{\$} \texttt{KeyGen}(1^\lambda)$
$(m_0, m_1) \xleftarrow{\$} A^{\text{ATK}}(1^\lambda, \texttt{pk})$
$b \xleftarrow{\$} \{0, 1\}$
$c^* \xleftarrow{\$} \texttt{Enc}(\texttt{pk}, m_b)$
$\hat{b} \xleftarrow{\$} A^{\text{ATK}}(1^\lambda, \texttt{pk}, c^*)$
return $[\![\hat{b} = b]\!]$

Figure A.2: IND-ATK game: adversary is asked to distinguish the encryption of one message from another

In the IND-ATK game (Figure A.2), the adversary chooses two distinct messages and receives the encryption of one of them, randomly selected by the challenger. The advantage of an adversary is its probability of correctly distinguishing the ciphertext of one message

from the other beyond blind guess: $\mathrm{Adv}^{\text{IND-ATK}}_{\mathtt{PKE}}(A) = |P[\hat{b} = b] - \frac{1}{2}|$. A PKE is said to be IND-ATK secure if no efficient adversary can win the IND-ATK game with non-negligible advantage.

$\mathcal{O}^{\mathtt{Dec}}(c)$
1: **return** $\mathtt{Dec}(\mathtt{sk}, c)$

$\mathcal{O}^{\mathtt{PCO}}(m, c)$
1: **return** $[\![\mathtt{Dec}(\mathtt{sk}, c) = m]\!]$

Figure A.3: Decryption oracle $\mathcal{O}^{\mathtt{Dec}}$ (left) answers decryption queries by returning the decryption of the queried ciphertext. Plaintext-checking oracle $\mathcal{O}^{\mathtt{PCO}}$ (right) answers whether the queried plaintext is the decryption of the queried ciphertext.

In public-key cryptography, all adversaries are assumed to have access to the public key (ATK = CPA). If the adversary has access to a decryption oracle, it is said to mount chosen-ciphertext attack (ATK = CCA). If the adversary has access to a plaintext-checking oracle (PCO) , then it is said to mount plaintext-checking attack (ATK = PCA). See Figure A.3 for algorithmic description of the oracles.

$$\text{ATK} = \begin{cases} \text{CPA} & \mathcal{O}^{\text{ATK}} = \cdot \\ \text{PCA} & \mathcal{O}^{\text{ATK}} = \mathcal{O}^{\mathtt{PCO}} \\ \text{CCA} & \mathcal{O}^{\text{ATK}} = \mathcal{O}^{\mathtt{Dec}} \end{cases}$$

A.2 Key encapsulation mechanism (KEM)

Syntax A key encapsulation mechanism (KEM) is a collection of three routines $(\mathtt{KeyGen}, \mathtt{Encap}, \mathtt{Decap})$ defined over some ciphertext space $\mathcal{C}$ and some key space $\mathcal{K}$. Key generation $\mathtt{KeyGen} : 1^\lambda \mapsto (\mathtt{pk}, \mathtt{sk})$ is a randomized routine that returns a keypair. Encapsulation $\mathtt{Encap} : \mathtt{pk} \mapsto (c, K)$ is a randomized routine that takes a public encapsulation key and returns a pair of ciphertext c and shared secret K (also commonly referred to as session key). Decapsulation $\mathtt{Decap} : (\mathtt{sk}, c) \mapsto K$ is a deterministic routine that uses the secret key $\mathtt{sk}$ to recover the shared secret K from the input ciphertext c. Where the KEM chooses to reject invalid ciphertext explicitly, the decapsulation routine can also output the rejection symbol $\perp$. We assume a KEM to be perfectly correct:

$$P\left[\mathtt{Decap}(\mathtt{sk}, c) = K \mid (\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda); (c, K) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk})\right] = 1$$

Security Similar to PKE security, the security of KEM is discussed using adversarial games. In the IND-ATK game (Figure A.4), the challenger generates a random keypair and encapsulates a random secret; the adversary is given the public key and the ciphertext, then asked to distinguish the shared secret from a random bit string.

KEM IND-ATK Game

1: $(\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}((1^\lambda))$
2: $(c^*, K_0) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk})$
3: $K_1 \xleftarrow{\$} \mathcal{K}$
4: $b \xleftarrow{\$} \{0,1\}$
5: $\hat{b} \xleftarrow{\$} A^{\mathtt{ATK}}(1^\lambda, \mathtt{pk}, c^*, K_b)$
6: **return** $[\![\hat{b} = b]\!]$

Figure A.4: The IND-ATK game for KEM

The advantage of an adversary is its probability of winning beyong blind guess. A KEM is said to be IND-ATK secure if no efficient adversary can win the IND-ATK game with non-negligible advantage.

$$\mathtt{Adv}^{\mathtt{IND\text{-}ATK}}(A) = \left| P\left[A^{\mathtt{ATK}}(1^\lambda, c^*, K_b) = b \mid \begin{array}{r} (\mathtt{pk}, \mathtt{sk}) \xleftarrow{\$} \mathtt{KeyGen}(1^\lambda); \\ (c^*, K_0) \xleftarrow{\$} \mathtt{Encap}(\mathtt{pk}); \\ K_1 \xleftarrow{\$} \mathcal{K}; b \xleftarrow{\$} \{0,1\} \end{array} \right] - \frac{1}{2} \right|$$

By default, all adversaries are assumed to have the public key, with which they can mount chosen plaintext attacks (ATK = CPA). If the adversary as access to a decapsulation oracle $\mathcal{O}^{\mathtt{Decap}} : c \mapsto \mathtt{Decap}(\mathtt{sk}, c)$, it is said to mount a chosen-ciphertext attack (ATK = CCA).

A.3 Message authentication code (MAC)

Syntax A message authentication code (MAC) is a collection of two routines `Sign` and `Verify` defined over some key space $\mathcal{K}$, some message space $\mathcal{M}$, and some tag space $\mathcal{T}$. The signing routine $\mathtt{Sign} : (k, m) \mapsto t$ authenticates the message m under the symmetric

key k by producing a tag t. The verification routine $\texttt{Verify}(k, m, t)$ outputs 1 if the message-tag pair (m, t) is authentic under the symmetric key k and 0 otherwise. Many MAC constructions are deterministic: for these constructions it is simpler to denote the signing routine by $t \leftarrow \texttt{MAC}(k, m)$, and verification done using a simple comparison. Some MAC constructions require a distinct or randomized nonce $r \xleftarrow{\$} \mathcal{R}$, and the signing routine will take this additional argument $t \leftarrow \texttt{MAC}(k, m; r)$.

Security The standard security notion for a MAC is *existential unforgeability under chosen message attack (EUF-CMA)*. We define it using an adversarial game in which an adversary has access to a signing oracle $\mathcal{O}^{\texttt{Sign}} : m \mapsto \texttt{Sign}(k, m)$ and tries to produce a valid message-tag pair that has not been queried from the signing oracle (Figure A.5).

MAC EUF-CMA game

1: $k^* \xleftarrow{\$} \mathcal{K}$
2: $(\hat{m}, \hat{t}) \xleftarrow{\$} A^{\texttt{CMA}}()$
3: **return** $[\![\texttt{Verify}(k^*, \hat{m}, \hat{t}) \wedge (\hat{m}, \hat{t}) \notin \mathcal{O}^{\texttt{Sign}}]\!]$

Figure A.5: The signing oracle signs the queried message with the secret key. The adversary must produce a message-tag pair that has never been queried before

The advantage of the adversary is the probability that it successfully produces a valid message-tag pair. A MAC is said to be EUF-CMA secure if no efficient adversary has non-negligible advantage. Some MACs are *one-time existentially unforgeable* (we call them one-time MAC), meaning that each secret key can be used to authenticate exactly one message. The corresponding security game is identical to the EUF-CMA game except for that the signing oracle will only answer up to one query.